

# Compte Rendu — TP1 Kubernetes

---

**Binôme : Corentin GODON & Matthias DAUVEL**

**Module : Arthur BARADEL — KUBERNETES**

**Date : 04 Mai 2026**

---

## Pré-TP — Construction des images Docker du fil rouge

### Objectif

Construire et publier les images Docker du frontend (nginx) et du backend (Flask) sur Docker Hub, afin qu'elles soient accessibles depuis le cluster k3s sans `imagePullSecrets`.

### Architecture cible

- `webapp-backend` : API Flask exposant `/api/messages` (GET/POST) et `/api/health`
- `webapp-frontend` : nginx servant une page HTML statique qui appelle l'API via un proxy `/api/`

### Ce que nous avons fait

Nous avons créé la structure de fichiers suivante :

```
webapp/
├── frontend/
│   ├── Dockerfile
│   ├── nginx.conf
│   └── html/
│       ├── index.html
│       └── app.js
└── backend/
    ├── Dockerfile
    ├── requirements.txt
    └── app.py
```

Le `nginx.conf` configure un proxy crucial vers le backend — le nom `backend-svc` est résolu par CoreDNS en TP1, et sera remplacé par un Ingress en TP2 :

```
location /api/ {
    proxy_pass http://backend-svc:5000/api/;
    proxy_set_header Host $host;
}
```

### Build et publication

```

$env:DOCKERHUB_USER = "mdprogra"
$env:TAG = "v1.0"

docker build --platform linux/amd64 -t docker.io/$env:DOCKERHUB_USER/webapp-backend:$env:TAG ./backend
docker push docker.io/$env:DOCKERHUB_USER/webapp-backend:$env:TAG

docker build --platform linux/amd64 -t docker.io/$env:DOCKERHUB_USER/webapp-frontend:$env:TAG ./frontend
docker push docker.io/$env:DOCKERHUB_USER/webapp-frontend:$env:TAG

```

## Problème rencontré — Architecture AMD64

Lors du premier build, les images ont été buildées sans préciser la plateforme cible. Les VMs Scaleway étant en `linux/amd64`, les pods tombaient en `ImagePullBackOff` avec l'erreur :

```
no match for platform in manifest: not found
```

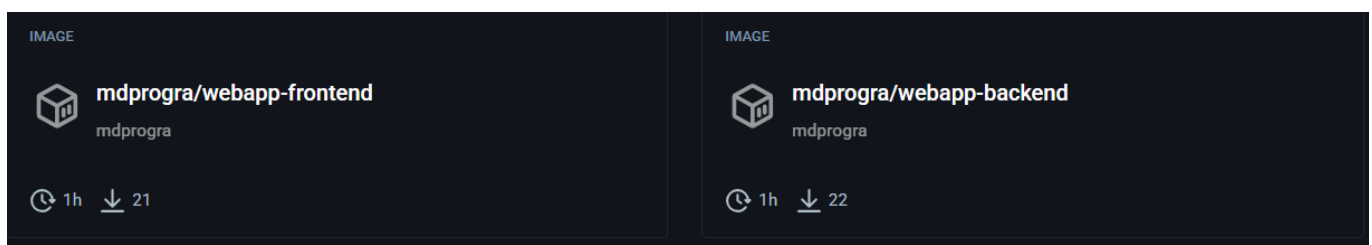
**Solution** : rebuild en forçant explicitement la plateforme AMD64 avec `docker buildx` :

```

docker buildx create --use --name mybuilder2
docker buildx inspect --bootstrap
docker buildx build --platform linux/amd64 `
  -t mdprogra/webapp-frontend:v1.2 `
  ./frontend --push --no-cache --progress=plain
docker buildx build --platform linux/amd64 `
  -t mdprogra/webapp-backend:v1.2 `
  ./backend --push --no-cache --progress=plain

```

Les images sont publiées en **public** sur Docker Hub : <https://hub.docker.com/u/mdprogra>



## Bloc 1 — Installation du cluster k3s

### Objectif

Installer un cluster k3s multi-nœuds (1 server + 2 agents) et vérifier que tous les nœuds et les pods système sont opérationnels.

## Infrastructure Scaleway

| VM   | Hostname          | IP publique    | Rôle          |
|------|-------------------|----------------|---------------|
| VM 1 | GODON-k3s-server  | 212.47.230.56  | Control plane |
| VM 2 | GODON-k3s-agent-1 | 163.172.161.25 | Worker 1      |
| VM 3 | GODON-k3s-agent-2 | 212.47.246.29  | Worker 2      |

Toutes les VMs sont sous Ubuntu 26.04, zone PAR-1. Ports 6443/TCP et 8472/UDP ouverts entre les VMs.

### Étape 1.1 — Installation du server

Sur **GODON-k3s-server** :

```
curl -sfL https://get.k3s.io | sh -
sudo systemctl status k3s
sudo kubectl get nodes
```

Récupération du token de jonction :

```
sudo cat /var/lib/rancher/k3s/server/node-token
```

### Étape 1.2 — Jonction des agents

Sur **chaque agent** (remplacer <TOKEN>) :

```
curl -sfL https://get.k3s.io | \
  K3S_URL=https://212.47.230.56:6443 \
  K3S_TOKEN=<TOKEN> \
  sh -
sudo systemctl status k3s-agent
```

### Étape 1.3 — Configuration de kubectl

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
chmod 600 ~/.kube/config
echo 'export KUBECONFIG=~/.kube/config' >> ~/.bashrc
source ~/.bashrc
```

### Étape 1.4 — Vérification du cluster

```
kubectl get nodes -o wide
kubectl get pods -A
```

Résultat :

| NAME              | STATUS | ROLES         | AGE   | VERSION      |
|-------------------|--------|---------------|-------|--------------|
| godon-k3s-agent-1 | Ready  | <none>        | 4m15s | v1.35.4+k3s1 |
| godon-k3s-agent-2 | Ready  | <none>        | 3m26s | v1.35.4+k3s1 |
| godon-k3s-server  | Ready  | control-plane | 6m34s | v1.35.4+k3s1 |

Name

IP address

Created

Zone

|                                                                                                   |                |                        |       |
|---------------------------------------------------------------------------------------------------|----------------|------------------------|-------|
| <div><div></div><div></div></div> <div><div>GODON-k3s-agent-2</div><div>BASIC2-A4C-8G</div></div> | 212.47.246.29  | less than a minute ago | PAR 1 |
| <div><div></div><div></div></div> <div><div>GODON-k3s-agent-1</div><div>BASIC3-X4C-8G</div></div> | 163.172.161.25 | 1 minute ago           | PAR 1 |
| <div><div></div><div></div></div> <div><div>GODON-k3s-server</div><div>BASIC3-X2C-8G</div></div>  | 212.47.230.56  | 1 minute ago           | PAR 1 |

```
ubuntu@GODON-k3s-server:~$ sudo kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-RUNTIME
godon-k3s-agent-1 Ready <none> 73s v1.35.4+k3s1 163.172.161.25 <none> Ubuntu 26.04 LTS 7.0.0-14-generic containerd://2.2.3-k3s1
godon-k3s-agent-2 Ready <none> 24s v1.35.4+k3s1 212.47.246.29 <none> Ubuntu 26.04 LTS 7.0.0-14-generic containerd://2.2.3-k3s1
godon-k3s-server Ready control-plane 3m32s v1.35.4+k3s1 212.47.230.56 <none> Ubuntu 26.04 LTS 7.0.0-14-generic containerd://2.2.3-k3s1
```

```
ubuntu@GODON-k3s-server:~$ kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-system coredns-c4dbffb5f-mpxp2 1/1 Running 0 8m11s
kube-system helm-install-traefik-55ctt 0/1 Completed 1 8m1s
kube-system helm-install-traefik-crd-bbwsr 0/1 Completed 0 8m1s
kube-system local-path-provisioner-5c4dc5d66d-tkl2s 1/1 Running 0 8m10s
kube-system metrics-server-786d997795-kpmtg 1/1 Running 0 8m6s
kube-system svclb-traefik-361a7d80-7m77k 2/2 Running 0 7m47s
kube-system svclb-traefik-361a7d80-qflkk 2/2 Running 0 5m8s
kube-system svclb-traefik-361a7d80-tddmr 2/2 Running 0 5m56s
kube-system traefik-9bcd9bd9-jzpk6 1/1 Running 0 7m47s
```

☒ **Checkpoint 1.A :** 3 nœuds en **Ready**. Pods système (**coredns**, **traefik**, **local-path-provisioner**, **metrics-server**) en **Running**.

## Bloc 2 — Pods, ReplicaSets, Deployments

### Objectif

Comprendre et utiliser les primitives Pod, ReplicaSet, Deployment. Observer l'auto-réparation, le scaling déclaratif et le rolling update.

### Étape 2.1 — Pod impératif & déclaratif

#### 1. Pod impératif :

Test de création rapide d'un pod en ligne de commande :

```

ubuntu@GODON-k3s-server:~$ kubectl run nginx-test --image=nginx:1.27-alpine --port=80
pod/nginx-test created
ubuntu@GODON-k3s-server:~$ kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
backend-58d789f58c-mcxcs             0/1     ImagePullBackOff    0           74m
backend-58d789f58c-wjlgx             1/1     Running             0           74m
backend-64b7bdb4f9-hz2r8             1/1     Running             0           97m
backend-init-6c6c9db4fd-6t5gp        0/1     Init:0/1            0           6m12s
curl-test                            1/1     Running             0           54m
frontend-5f6fd4c584-h8z76            1/1     Running             0           96m
frontend-5f6fd4c584-hf7pv            1/1     Running             0           83m
frontend-668b748777-7xqcn            0/1     Running             21 (7s ago)  61m
nginx-test                           1/1     Running             0           4s
ubuntu@GODON-k3s-server:~$ kubectl exec -it nginx-test -- sh
/ # curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ # exit

```

## 2. Pod déclaratif (01-pod.yaml) :

Création d'un pod nginx basique pour comprendre la structure d'un manifest YAML et observer le comportement sans Deployment :

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: demo
spec:
  containers:
  - name: nginx
    image: nginx:1.27-alpine
    ports:
    - containerPort: 80
  resources:
    requests: { cpu: "50m", memory: "64Mi" }
    limits: { cpu: "200m", memory: "128Mi" }

```

```

kubectl apply -f 01-pod.yaml
kubectl get pods -o wide
kubectl describe pod nginx-pod
kubectl exec -it nginx-pod -- sh
kubectl delete -f 01-pod.yaml

```

Un Pod seul supprimé n'est **pas recréé** — il n'existe pas de contrôleur pour le surveiller. C'est le rôle du Deployment/ReplicaSet.

```

ubuntu@GODON-k3s-server:~$ cat > 01-pod.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: demo
spec:
  containers:
  - name: nginx
    image: nginx:1.27-alpine
    ports:
    - containerPort: 80
    resources:
      requests: { cpu: "50m", memory: "64Mi" }
      limits: { cpu: "200m", memory: "128Mi" }
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 01-pod.yaml
pod/nginx-pod created
ubuntu@GODON-k3s-server:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx-pod 0/1     ContainerCreating  0          4s

```

## Étape 2.2 — Deployment frontend (02-frontend-deploy.yaml)

Un Deployment gère un ReplicaSet qui maintient le nombre souhaité de pods en vie. Les `readinessProbe` et `livenessProbe` permettent à Kubernetes de vérifier la santé des pods :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels: { app: webapp, tier: front }
spec:
  replicas: 3
  selector:
    matchLabels: { app: webapp, tier: front }
  template:
    metadata:
      labels: { app: webapp, tier: front }
    spec:
      containers:
      - name: frontend
        image: docker.io/mdprogra/webapp-frontend:v1.2
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet: { path: /, port: 80 }
          initialDelaySeconds: 2
        livenessProbe:
          httpGet: { path: /, port: 80 }
          initialDelaySeconds: 5
          periodSeconds: 10

```

```

kubectl apply -f 02-frontend-deploy.yaml
kubectl get deploy,rs,pod
kubectl rollout status deploy/frontend

```

```

ubuntu@GODON-k3s-server:~$ cat > 02-frontend-deploy.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels: { app: webapp, tier: front }
spec:
  replicas: 3
  selector:
    matchLabels: { app: webapp, tier: front }
  template:
    metadata:
      labels: { app: webapp, tier: front }
    spec:
      containers:
        - name: frontend
          image: docker.io/mdprogra/webapp-frontend:v1.0
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet: { path: /, port: 80 }
            initialDelaySeconds: 2
          livenessProbe:
            httpGet: { path: /, port: 80 }
            initialDelaySeconds: 5
            periodSeconds: 10
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 02-frontend-deploy.yaml
deployment.apps/frontend created
ubuntu@GODON-k3s-server:~$ kubectl get deploy,rs,pod
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/frontend            0/3      3              0             4s

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/frontend-599f5cf46b 3           3          0        4s

NAME                                READY    STATUS              RESTARTS    AGE
pod/frontend-599f5cf46b-4mLnF        0/1      ContainerCreating    0            4s
pod/frontend-599f5cf46b-892xg        0/1      ContainerCreating    0            4s
pod/frontend-599f5cf46b-x7clw        0/1      ErrImagePull         0            4s

```

## Étape 2.3 — Auto-réparation

Kubernetes surveille en permanence l'état du cluster via le ReplicaSet controller. Si un pod est supprimé manuellement, il est immédiatement recréé :

```

kubectl delete pod frontend-5f6fd4c584-8xbs5
kubectl get pods -l tier=front -w

```

On observe la création du pod de remplacement (**hf7pv**) quasi-instantanément.

```

ubuntu@GODON-k3s-server:~$ kubectl get pods -l tier=front
NAME                                READY    STATUS              RESTARTS    AGE
frontend-5f6fd4c584-6bhj9          0/1      ImagePullBackOff    0            12m
frontend-5f6fd4c584-8xbs5          1/1      Running             0            12m
frontend-5f6fd4c584-h8z76          1/1      Running             0            12m
ubuntu@GODON-k3s-server:~$ kubectl delete pod frontend-5f6fd4c584-8xbs5
pod "frontend-5f6fd4c584-8xbs5" deleted from default namespace
ubuntu@GODON-k3s-server:~$ kubectl get pods -l tier=front -w
NAME                                READY    STATUS              RESTARTS    AGE
frontend-5f6fd4c584-6bhj9          0/1      ImagePullBackOff    0            12m
frontend-5f6fd4c584-h8z76          1/1      Running             0            12m
frontend-5f6fd4c584-hf7pv         0/1      Running             0            3s
frontend-5f6fd4c584-hf7pv         1/1      Running             0            11s

```

## Étape 2.4 — Scaling

```

# Scaling impératif
kubectl scale deploy/frontend --replicas=5
kubectl get pods -l tier=front

kubectl scale deploy/frontend --replicas=2

```

Le scaling est **déclaratif** : on déclare l'état souhaité et Kubernetes s'assure de le respecter. La même opération peut être faite en modifiant **replicas** dans le YAML puis **kubectl apply**.

```
ubuntu@GODON-k3s-server:~$ kubectl scale deploy/frontend --replicas=5
deployment.apps/frontend scaled
ubuntu@GODON-k3s-server:~$ kubectl get pods -l tier=front
NAME                                READY   STATUS              RESTARTS   AGE
frontend-5f6fd4c584-6bhj9           0/1     ImagePullBackOff    0           35m
frontend-5f6fd4c584-h8z76           1/1     Running             0           35m
frontend-5f6fd4c584-hf7pv           1/1     Running             0           22m
frontend-5f6fd4c584-rwlp6           0/1     ErrImagePull        0           5s
frontend-668b748777-7xqcn           0/1     Running             0           5s
frontend-668b748777-bhdL7           0/1     Running             0           5s
frontend-668b748777-pjsvp           0/1     ImagePullBackOff    0           20m
ubuntu@GODON-k3s-server:~$ kubectl scale deploy/frontend --replicas=2
deployment.apps/frontend scaled
```

## Étape 2.5 — Rolling update & Rollback

```
# Mise à jour de l'image vers v1.1
kubectl set image deploy/frontend frontend=docker.io/mdprogra/webapp-frontend:v1.1
kubectl rollout status deploy/frontend
kubectl rollout history deploy/frontend

# Rollback à la version précédente
kubectl rollout undo deploy/frontend
```

Le rolling update remplace les pods progressivement **sans interruption de service**. Le rollback revient à la révision précédente en utilisant l'historique des ReplicaSets.

```
ubuntu@GODON-k3s-server:~$ kubectl get pods -l tier=front -w
NAME                                READY   STATUS              RESTARTS   AGE
frontend-5f6fd4c584-6bhj9           0/1     ImagePullBackOff    0           16m
frontend-5f6fd4c584-h8z76           1/1     Running             0           16m
frontend-5f6fd4c584-hf7pv           1/1     Running             0           3m53s
frontend-668b748777-pjsvp           0/1     ImagePullBackOff    0           2m15s
```

```
ubuntu@GODON-k3s-server:~$ kubectl rollout undo deploy/frontend
deployment.apps/frontend rolled back
ubuntu@GODON-k3s-server:~$ kubectl rollout history deploy/frontend
deployment.apps/frontend
REVISION   CHANGE-CAUSE
3          <none>
4          <none>
```

☑ **Checkpoint 2** : 3 puis 5 puis 2 réplicas observés. Suppression manuelle d'un pod → création automatique. Rollback fonctionnel.

## Bloc 3 — Services et exposition

### Objectif

Exposer les pods via des Services pour permettre la communication interne (ClusterIP) et l'accès depuis l'extérieur du cluster (NodePort).

### Étape 3.1 — Service ClusterIP (03-frontend-svc.yaml)

Un Service ClusterIP expose les pods **uniquement à l'intérieur du cluster**, avec une IP virtuelle stable indépendante du cycle de vie des pods. Le selector fait le lien avec les pods via leurs labels :

```
apiVersion: v1
kind: Service
metadata:
```



```

    name: frontend-svc
spec:
  type: ClusterIP
  selector: { app: webapp, tier: front }
  ports:
    - port: 80
      targetPort: 80

```

```

kubectl apply -f 03-frontend-svc.yaml
kubectl get svc
kubectl describe svc frontend-svc

# Test depuis un pod éphémère
kubectl run curl-test --rm -it --image=curlimages/curl:latest -- sh
# curl http://frontend-svc/

```

```

ubuntu@GODON-k3s-server:~$ cat > 03-frontend-svc.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: ClusterIP
  selector: { app: webapp, tier: front }
  ports:
    - port: 80
      targetPort: 80
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 03-frontend-svc.yaml
service/frontend-svc unchanged
ubuntu@GODON-k3s-server:~$ kubectl get svc

```

| NAME         | TYPE      | CLUSTER-IP    | EXTERNAL-IP | PORT(S)  | AGE  |
|--------------|-----------|---------------|-------------|----------|------|
| backend-svc  | ClusterIP | 10.43.51.23   | <none>      | 5000/TCP | 116s |
| frontend-svc | ClusterIP | 10.43.163.155 | <none>      | 80/TCP   | 32s  |
| kubernetes   | ClusterIP | 10.43.0.1     | <none>      | 443/TCP  | 51m  |

```

ubuntu@GODON-k3s-server:~$ kubectl run curl-test --rm -it --image=curlimages/curl:latest -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
~ $ curl http://frontend-svc/
<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>K8s Guestbook</title></head>
<body>
<h1>Livre d'or Kubernetes</h1>
<p>Servi par : <span id="host"></span></p>
<ul id="list"></ul>
<input id="msg" placeholder="Votre message"/>
<button onclick="post()">Envoyer</button>
<script src="app.js"></script>
</body>
</html>~ $

```

## Étape 3.2 — Service NodePort (04-frontend-nodeport.yaml)

Un Service NodePort expose les pods **depuis l'extérieur du cluster** sur un port fixe (30080) de chaque nœud :

```

apiVersion: v1
kind: Service
metadata:
  name: frontend-nodeport
spec:
  type: NodePort
  selector: { app: webapp, tier: front }
  ports:
    - port: 80

```

```
targetPort: 80
nodePort: 30080
```

Le frontend est accessible sur `http://212.47.230.56:30080`. À ce stade, le frontend s'affiche mais l'API est cassée — le backend n'est pas encore déployé.

```
ubuntu@GODON-k3s-server:~$ cat > 04-frontend-nodeport.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: frontend-nodeport
spec:
  type: NodePort
  selector: { app: webapp, tier: front }
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 04-frontend-nodeport.yaml
service/frontend-nodeport created
ubuntu@GODON-k3s-server:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
backend-svc         ClusterIP   10.43.51.23    <none>          5000/TCP         3m12s
frontend-nodeport   NodePort    10.43.143.180  <none>          80:30080/TCP     5s
frontend-svc        ClusterIP   10.43.163.155  <none>          80/TCP           108s
kubernetes           ClusterIP   10.43.0.1      <none>          443/TCP          52m
```

☑ **Checkpoint 3** : Service `ClusterIP` joignable depuis un pod du cluster. Frontend accessible sur `:30080` depuis le réseau.

## Bloc 4 — Application multi-tier complète

### Objectif

Déployer le backend Flask, le connecter au frontend via la résolution DNS interne de Kubernetes (CoreDNS), et valider le fonctionnement de bout en bout.

### Étape 4.1 — Backend (`05-backend.yaml`)

Le fichier contient le Deployment et le Service dans un seul manifest (séparés par `---`).

⚠ Le nom du Service `backend-svc` doit correspondre exactement à celui codé dans `nginx.conf` (`proxy_pass http://backend-svc:5000`).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels: { app: webapp, tier: back }
spec:
  replicas: 2
  selector:
    matchLabels: { app: webapp, tier: back }
  template:
    metadata:
      labels: { app: webapp, tier: back }
    spec:
      containers:
      - name: backend
        image: docker.io/mdprogra/webapp-backend:v1.2
        ports:
```

```

    - containerPort: 5000
  env:
    - name: APP_ENV
      value: "tp1"
  readinessProbe:
    httpGet: { path: /api/health, port: 5000 }
    initialDelaySeconds: 3
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector: { app: webapp, tier: back }
  ports:
    - port: 5000
      targetPort: 5000

```

```

kubectl apply -f 05-backend.yaml
kubectl get pods,svc -l app=webapp

```

## Étape 4.2 — Résolution DNS interne (CoreDNS)

Le nom `backend-svc` dans `nginx.conf` est résolu automatiquement par CoreDNS. Chaque Service Kubernetes reçoit un nom DNS au format :

`<nom-service>.<namespace>.svc.cluster.local`

```

kubectl exec -it <pod-frontend> -- sh
nslookup backend-svc
# → backend-svc.default.svc.cluster.local → 10.43.51.23
wget -qO- http://backend-svc:5000/api/health

```

```

root@ubuntu:~# kubectl exec -it frontend-5f6fd4c384-8xb55 -- sh
/ # nslookup backend-svc
Server:      10.43.0.10
Address:     10.43.0.10:53

** server can't find backend-svc.cluster.local: NXDOMAIN
Name:   backend-svc.default.svc.cluster.local
Address: 10.43.51.23

** server can't find backend-svc.cluster.local: NXDOMAIN
** server can't find backend-svc.svc.cluster.local: NXDOMAIN
** server can't find backend-svc.svc.cluster.local: NXDOMAIN

```

## Étape 4.3 — Test end-to-end

L'application est accessible sur `http://212.47.230.56:30080`. Le livre d'or affiche le nom du pod backend qui répond (`served_by`), ce qui change selon le pod sélectionné par le load balancer — preuve du fonctionnement multi-tier.

```
kubectl logs -l tier=back -f
```

# Livre d'or Kubernetes

Servi par : backend-64b7bdb4f9-hz2r8

- Bienvenue sur le livre d'or k8s !

## Problème rencontré — ImagePullBackOff sur agent-2

Le nœud `godon-k3s-agent-2` présentait systématiquement des erreurs `ImagePullBackOff`. Diagnostic :

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod>
# → Events: Failed to pull image: no match for platform in manifest
kubectl get endpoints backend-svc
```

**Cause** : images buildées sans `--platform linux/amd64`.

**Solution** : rebuild avec `docker buildx --platform linux/amd64` (cf. Pré-TP).

Le nœud `agent-2` continuait à avoir des problèmes après correction, probablement dû à un cache containerd corrompu. Les pods se sont répartis sur `agent-1` et `server` sans impact sur le fonctionnement global.

☒ **Checkpoint 4** : Application multi-tier fonctionnelle de bout en bout. Au moins une panne diagnostiquée et corrigée.

---

## Bloc 5 — Défis ouverts

### Défi A — Anti-affinité des pods backend

#### Objectif

Garantir que deux pods backend ne soient **jamais schedulés sur le même nœud**. Cela améliore la haute disponibilité : si un nœud tombe, au moins un pod backend reste disponible sur un autre nœud.

#### Concept

L'anti-affinité (`podAntiAffinity`) dit au scheduler Kubernetes : *"ne place pas ce pod sur un nœud où tourne déjà un pod avec ces labels"*.

Il existe deux modes :

- `requiredDuringSchedulingIgnoredDuringExecution` — **strict** : le pod reste en `Pending` si aucun nœud libre
- `preferredDuringSchedulingIgnoredDuringExecution` — **souple** : préférence, non bloquant

Nous utilisons le mode **strict** avec `topologyKey: kubernetes.io/hostname` pour séparer les pods par nœud physique.

### YAML (`defis/defi-A.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels: { app: webapp, tier: back }
spec:
  replicas: 2
  selector:
    matchLabels: { app: webapp, tier: back }
  template:
    metadata:
      labels: { app: webapp, tier: back }
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchLabels:
                  app: webapp
                  tier: back
              topologyKey: kubernetes.io/hostname
      containers:
        - name: backend
          image: docker.io/mdprogra/webapp-backend:v1.2
          ports:
            - containerPort: 5000
          env:
            - name: APP_ENV
              value: "tp1"
          readinessProbe:
            httpGet: { path: /api/health, port: 5000 }
            initialDelaySeconds: 3
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
```

```
selector: { app: webapp, tier: back }
ports:
- port: 5000
  targetPort: 5000
```

## Vérification

```
kubectl get pods -o wide -l tier=back
```

Résultat obtenu :

| NAME                     | READY | STATUS  | NODE              |
|--------------------------|-------|---------|-------------------|
| backend-58d789f58c-wjlgx | 1/1   | Running | godon-k3s-server  |
| backend-64b7bdb4f9-hz2r8 | 1/1   | Running | godon-k3s-agent-1 |

Les deux pods backend sont bien sur des **nœuds différents** ☒

```
ubuntu@godon-k3s-server:~$ kubectl get pods -o wide -l tier=back
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE               NOMINATED NODE   READINESS GATES
backend-58d789f58c-wjlgx 1/1     Running   0           16s   10.42.0.14    godon-k3s-server   <none>           <none>
backend-64b7bdb4f9-hz2r8 1/1     Running   0           22m   10.42.1.7     godon-k3s-agent-1  <none>           <none>
backend-64b7bdb4f9-xfpm 0/1     Terminating 0           22m   10.42.2.9     godon-k3s-agent-2  <none>           <none>
```

☒ **Défi A validé** : l'anti-affinité garantit la répartition des pods backend sur des nœuds distincts.

## Défi B — Init container

### Objectif

Ajouter un **initContainer** au backend qui simule l'attente d'une base de données (**postgres-svc**) avant de démarrer l'application principale. Cela montre comment bloquer le démarrage d'un pod tant qu'une dépendance n'est pas prête.

### Vérification et Logs

L'application backend reste en **Init:0/1** tant que l'initContainer tourne. Une fois terminé, le pod passe en **Running**.

```
backend-init-6c6c9db4fd-6t5gp 0/1 Init:0/1 0 3s
```

```

^Cubuntu@GODON-k3s-server:~$ kubectl logs backend-init-6c6c9db4fd-6t5gp -c wait-for-postgres
Server:      10.43.0.10
Address:     10.43.0.10:53

** server can't find postgres-svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.default.svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.default.svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.svc.cluster.local: NXDOMAIN

En attente de postgres-svc...
Server:      10.43.0.10
Address:     10.43.0.10:53

** server can't find postgres-svc.default.svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.default.svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.svc.cluster.local: NXDOMAIN

En attente de postgres-svc...
Server:      10.43.0.10
Address:     10.43.0.10:53

** server can't find postgres-svc.cluster.local: NXDOMAIN
** server can't find postgres-svc.svc.cluster.local: NXDOMAIN

```

☒ **Défi B validé** : l'initContainer s'exécute correctement et loggue son statut avant de laisser le conteneur principal démarrer.

## Défi C — Multi-conteneurs (Sidecar)

### Objectif

Modifier le pod frontend pour ajouter un conteneur *sidecar* (*busybox*) qui lit en temps réel (*tail -f*) les logs de nginx via un volume partagé.

### Vérification et Logs

Le pod frontend s'affiche désormais avec 2/2 conteneurs prêts (le serveur nginx + le sidecar).

```

ubuntu@GODON-k3s-server:~$ kubectl get pods -w
NAME                                READY   STATUS              RESTARTS   AGE
backend-5598f6dcd9-k94cc            0/1     Running             0           9s
backend-58d789f58c-wjlgx            1/1     Running             0           168m
backend-64b7bdb4f9-hz2r8            1/1     Running             0           3h10m
backend-init-6c6c9db4fd-6t5gp       0/1     Init:0/1            0           99m
curl-test                           1/1     Running             0           148m
frontend-5f6fd4c584-h8z76           1/1     Running             0           3h10m
frontend-5f6fd4c584-hf7pv           1/1     Running             0           177m
frontend-995d99765-z2pzx            0/1     Running             0           17s
frontend-sidecar-b67ccdf4-cl5w8     0/2     ImagePullBackOff    9 (2m57s ago) 10m
frontend-sidecar-cf5d58589-8fxjl    2/2     Running             0           4s
nginx-test                          1/1     Running             0           93m

```

En interrogeant spécifiquement les logs du deuxième conteneur, on obtient bien les logs d'accès nginx :

```

ubuntu@GODON-k3s-server:~$ kubectl logs frontend-sidecar-cf5d58589-8fxjl -c log-sidecar
10.42.2.15 - - [04/May/2026:12:30:02 +0000] "GET / HTTP/1.1" 200 355 "-" "curl/8.20.0" "-"

```

☒ **Défi C validé** : le conteneur sidecar tourne correctement dans le même pod que le frontend et accède aux logs partagés.

## Synthèse des commandes clés

| Commande                                                   | Description                                  |
|------------------------------------------------------------|----------------------------------------------|
| <code>kubectl apply -f &lt;fichier&gt;</code>              | Appliquer un manifest YAML                   |
| <code>kubectl get deploy,rs,pod</code>                     | Lister les ressources principales            |
| <code>kubectl get pods -o wide -w</code>                   | Observer les pods en temps réel avec le nœud |
| <code>kubectl describe pod &lt;pod&gt;</code>              | Détails et événements d'un pod               |
| <code>kubectl logs &lt;pod&gt; --previous</code>           | Logs du conteneur précédent (après crash)    |
| <code>kubectl exec -it &lt;pod&gt; -- sh</code>            | Shell dans un pod                            |
| <code>kubectl scale deploy/&lt;nom&gt; --replicas=N</code> | Scaling impératif                            |
| <code>kubectl rollout status deploy/&lt;nom&gt;</code>     | Suivre un déploiement                        |
| <code>kubectl rollout undo deploy/&lt;nom&gt;</code>       | Rollback                                     |
| <code>kubectl rollout history deploy/&lt;nom&gt;</code>    | Historique des révisions                     |
| <code>kubectl get events --sort-by=.lastTimestamp</code>   | Événements triés par date                    |
| <code>kubectl get endpoints &lt;svc&gt;</code>             | Vérifier les endpoints d'un Service          |
| <code>kubectl get svc</code>                               | Lister les services                          |

Pièges rencontrés

| Symptôme                         | Cause                                                       | Résolution                                                     |
|----------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|
| ImagePullBackOff                 | Images buildées sans <code>--platform linux/amd64</code>    | Rebuild avec <code>docker buildx --platform linux/amd64</code> |
| Service sans endpoints           | Selector ne correspond pas aux labels des pods              | Vérifier <code>kubectl describe svc</code> et les labels       |
| Frontend affiche mais API échoue | Backend non déployé ou <code>backend-svc</code> introuvable | Déployer le backend, vérifier le nom du Service                |
| Pod en Pending                   | Ressources insuffisantes ou anti-affinité stricte           | <code>kubectl describe pod</code> → Events                     |

Compte Rendu — TP2 Kubernetes

Binôme : Corentin GODON & Matthias DAUVEL  
Module : Arthur BARADEL — KUBERNETES  
Date : 04 Mai 2026

Pré-requis — Image backend v2.0



## Contexte

Le TP1 utilisait un backend Flask qui stockait les messages **en mémoire** (perdus à chaque redémarrage). Pour le TP2, le backend doit pouvoir se connecter à une base **PostgreSQL** via la variable d'environnement `DATABASE_URL`. Si celle-ci est absente, il retombe automatiquement en mode mémoire — ce qui assure la rétrocompatibilité avec le TP1.

## Modifications apportées

**backend/requirements.txt** — Ajout de `psycopg2-binary` :

```
flask==3.0.3
psycopg2-binary==2.9.9
```

**backend/app.py** — Ajout de la logique de connexion Postgres :

- Initialisation d'un pool de connexions via `psycopg2` si `DATABASE_URL` est définie
- Création automatique de la table `messages` au démarrage (`init_db()`)
- Le champ `backend_mode` du JSON retourne `"postgres"` ou `"memory"` selon le mode actif

## Build et push (multi-architecture)

Comme pour le TP1, on cible explicitement `linux/amd64` et `linux/arm64` pour couvrir les VMs Scaleway et les machines de développement (Mac M1/M2) :

```
$env:DOCKERHUB_USER = "mdprogra"

docker buildx build --platform linux/amd64,linux/arm64 \
-t docker.io/$env:DOCKERHUB_USER/webapp-backend:v2.0 \
./backend --push --no-cache
```

The screenshot shows the Docker Hub interface for the repository `mdprogra/webapp-backend`. The page is dark-themed. At the top, the repository name is displayed along with the user `mdprogra` and the update time 'Updated 32 minutes ago'. There is a 'Manage Repository' button in the top right. Below the repository name, there are icons for 'IMAGE', '0 stars', and '153 downloads'. The main content area is divided into two sections. On the left, under the 'Overview' tab, there is a message: 'No overview available. This repository doesn't have an overview'. On the right, under the 'Tags' tab, there is a 'Tag summary' section. It shows 'Recent tags' with a dropdown menu set to 'v2.0'. Below this, it lists 'Content type' as 'Image', 'Digest' as 'sha256:bbe0e9eb5...', 'Size' as '48.9 MB', and 'Last updated' as '32 minutes ago'. At the bottom of the right section, there is a command to pull the image: `docker pull mdprogra/webapp-backend:v2.0`.

☑ **Pré-requis validé** : l'image `mdprogra/webapp-backend:v2.0` est publiée en **public** sur Docker Hub, accessible pour les deux architectures.

## Bloc 1 — ConfigMaps & Secrets

### Objectif

Externaliser la configuration applicative et les credentials hors du code source. Comprendre la différence entre ConfigMap (données publiques) et Secret (données sensibles encodées en base64), et maîtriser leurs trois méthodes de création.

### Étape 1.1 — Namespace dédié

Pour isoler les ressources du TP2 du namespace `default` :

```
kubectl create namespace guestbook
kubectl config set-context --current --namespace=guestbook
kubectl config view --minify | grep namespace
```

Toutes les commandes suivantes opèrent dans ce namespace.

### Étape 1.2 — ConfigMap : trois méthodes de création

#### Méthode 1 — impérative (littéraux) :

```
kubectl create configmap demo-cm \
  --from-literal=APP_ENV=tp2 \
  --from-literal=LOG_LEVEL=info
kubectl get cm demo-cm -o yaml
kubectl delete cm demo-cm
```

```
ubuntu@GODON-k3s-server:~$ kubectl create namespace guestbook
namespace/guestbook created
ubuntu@GODON-k3s-server:~$ kubectl config set-context --current --namespace=guestbook
Context "default" modified.
ubuntu@GODON-k3s-server:~$ kubectl config view --minify | grep namespace
  namespace: guestbook
ubuntu@GODON-k3s-server:~$ kubectl create configmap demo-cm \
  --from-literal=APP_ENV=tp2 \
  --from-literal=LOG_LEVEL=info
configmap/demo-cm created
ubuntu@GODON-k3s-server:~$ kubectl get cm demo-cm -o yaml
apiVersion: v1
data:
  APP_ENV: tp2
  LOG_LEVEL: info
kind: ConfigMap
metadata:
  creationTimestamp: "2026-05-04T12:59:11Z"
  name: demo-cm
  namespace: guestbook
  resourceVersion: "11769"
  uid: 5cebbab6-9526-42e9-98a4-bd7bf21446bd
ubuntu@GODON-k3s-server:~$ kubectl delete cm demo-cm
configmap "demo-cm" deleted from guestbook namespace
```

#### Méthode 2 — à partir d'un fichier :

```
echo "welcome=Bienvenue sur le livre d'or persistant !" > app.properties
kubectl create configmap demo-cm --from-file=app.properties
```

```
kubectl get cm demo-cm -o yaml
kubectl delete cm demo-cm
```

```
ubuntu@GODON-k3s-server:~$ echo "welcome=Bienvenue sur le livre d'or persistant !" > app.properties
ubuntu@GODON-k3s-server:~$ kubectl create configmap demo-cm --from-file=app.properties
configmap/demo-cm created
ubuntu@GODON-k3s-server:~$ kubectl get cm demo-cm -o yaml
apiVersion: v1
data:
  app.properties: |
    welcome=Bienvenue sur le livre d'or persistant !
kind: ConfigMap
metadata:
  creationTimestamp: "2026-05-04T13:02:22Z"
  name: demo-cm
  namespace: guestbook
  resourceVersion: "11869"
  uid: 72f0148e-b8b1-4501-9ac2-9a2d565f4cd2
ubuntu@GODON-k3s-server:~$ kubectl delete cm demo-cm
configmap "demo-cm" deleted from guestbook namespace
```

### Méthode 3 — déclarative (privilégiée) via `10-configmap.yaml` :

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-config
data:
  APP_ENV: "tp2-prod"
  WELCOME_MESSAGE: "Bienvenue sur le livre d'or persistant !"
  LOG_LEVEL: "info"
```

```
kubectl apply -f 10-configmap.yaml
kubectl describe cm webapp-config
```

```
ubuntu@GODON-k3s-server:~$ cat > 10-configmap.yaml << 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-config
  namespace: guestbook
data:
  APP_ENV: "tp2-prod"
  WELCOME_MESSAGE: "Bienvenue sur le livre d'or persistant !"
  LOG_LEVEL: "info"
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 10-configmap.yaml
configmap/webapp-config created
ubuntu@GODON-k3s-server:~$ kubectl describe cm webapp-config
Name:         webapp-config
Namespace:    guestbook
Labels:       <none>
Annotations:  <none>

Data
====
APP_ENV:
----
tp2-prod
LOG_LEVEL:
----
info
WELCOME_MESSAGE:
----
Bienvenue sur le livre d'or persistant !

BinaryData
====
Events:  <none>
```

### Étape 1.3 — Secret : credentials Postgres

Le Secret est encodé en **base64**, pas chiffré — quiconque a accès à l'API Kubernetes peut décoder les valeurs. En production, il faudrait coupler avec **EncryptionConfiguration** côté kube-apiserver ou un gestionnaire externe (HashiCorp Vault, Sealed Secrets).

11-secret.yaml utilise `stringData` (Kubernetes encode lui-même) plutôt que `data` (qui exigerait du base64 manuel) :

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-credentials
type: Opaque
stringData:
  POSTGRES_USER: "guestbook"
  POSTGRES_PASSWORD: "ChangeMe_inTP2!"
  POSTGRES_DB: "guestbook"
```

```
kubectl apply -f 11-secret.yaml
kubectl get secret postgres-credentials -o yaml
echo "Z3Vlc3Rib29r" | base64 -d # → guestbook
```

```
ubuntu@GODON-k3s-server:~$ cat > 11-secret.yaml << 'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: postgres-credentials
  namespace: guestbook
type: Opaque
stringData:
  POSTGRES_USER: "guestbook"
  POSTGRES_PASSWORD: "ChangeMe_inTP2!"
  POSTGRES_DB: "guestbook"
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 11-secret.yaml
secret/postgres-credentials created
ubuntu@GODON-k3s-server:~$ kubectl get secret postgres-credentials -o yaml
apiVersion: v1
data:
  POSTGRES_DB: Z3Vlc3Rib29r
  POSTGRES_PASSWORD: Q2hhbmdlTWVfaW5UUDIh
  POSTGRES_USER: Z3Vlc3Rib29r
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":"postgres-credentials","namespace":"guestbook"},"stringData":{"POSTGRES_DB":"gu
estbook","POSTGRES_PASSWORD":"ChangeMe_inTP2!","POSTGRES_USER":"guestbook"},"type":"Opaque"}
  creationTimestamp: "2026-05-04T13:05:20Z"
  name: postgres-credentials
  namespace: guestbook
  resourceVersion: "11965"
  uid: 305eaa9c-5da6-4aed-a105-ebc0f53c5606
type: Opaque
ubuntu@GODON-k3s-server:~$ echo "Z3Vlc3Rib29r" | base64 -d
guestbookubuntu@GODON-k3s-server:~$ echo "Z3Vlc3Rib29r" | base64 -d
base64 -d
```

## Étape 1.4 — Injection de la config dans un pod test

Le pod `config-tester` (12-test-config.yaml) injecte le ConfigMap et le Secret comme variables d'environnement, et monte le ConfigMap en volume pour vérifier les deux modes d'accès :

```
apiVersion: v1
kind: Pod
metadata:
  name: config-tester
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "env | sort && sleep 3600"]
      envFrom:
```

```

- configMapRef:
  name: webapp-config
- secretRef:
  name: postgres-credentials
volumeMounts:
- name: cfg-vol
  mountPath: /etc/cfg
volumes:
- name: cfg-vol
  configMap:
    name: webapp-config

```

```

kubectl apply -f 12-test-config.yaml
kubectl logs config-tester | grep -E "APP_ENV|WELCOME|POSTGRES"
kubectl exec config-tester -- ls /etc/cfg
kubectl exec config-tester -- cat /etc/cfg/WELCOME_MESSAGE
kubectl delete -f 12-test-config.yaml

```

```

ubuntu@GODON-k3s-server:~$ cat > 12-test-config.yaml << 'EOF' << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: config-tester
  namespace: guestbook
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "env | sort && sleep 3600"]
    envFrom:
    - configMapRef:
        name: webapp-config
    - secretRef:
        name: postgres-credentials
    volumeMounts:
    - name: cfg-vol
      mountPath: /etc/cfg
  volumes:
  - name: cfg-vol
    configMap:
      name: webapp-config
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 12-test-config.yaml
pod/config-tester created
ubuntu@GODON-k3s-server:~$ kubectl logs config-tester | grep -E "APP_ENV|WELCOME|POSTGRES"
APP_ENV=tp2-prod
POSTGRES_DB=guestbook
POSTGRES_PASSWORD=ChangeMe_inTP2!
POSTGRES_USER=guestbook
WELCOME_MESSAGE=Bienvenue sur le livre d'or persistant !
ubuntu@GODON-k3s-server:~$ kubectl exec config-tester -- ls /etc/cfg
APP_ENV
LOG_LEVEL
WELCOME_MESSAGE
ubuntu@GODON-k3s-server:~$ kubectl exec config-tester -- cat /etc/cfg/WELCOME_MESSAGE
Bienvenue sur le livre d'or persistant !ubuntu@GODON-k3s-server:~$

```

```

ubuntu@GODON-k3s-server:~$ kubectl exec config-tester -- cat /etc/cfg/WELCOME_MESSAGE
Bienvenue sur le livre d'orkubectl delete -f 12-test-config.yaml-$ kubectl delete -f 12-test-config.yaml
pod "config-tester" deleted from guestbook namespace
kubectl get cm,secret -n guestbook
^Cubuntu@GODON-k3s-server:~$ kubectl get cm,secret -n guestbookok
NAME          DATA   AGE
configmap/kube-root-ca.crt  1       10m
configmap/webapp-config     3       4m34s

NAME          TYPE   DATA   AGE
secret/postgres-credentials  Opaque  3       3m16s

```

**Piège rencontré :** modifier un ConfigMap ne redémarre pas automatiquement les pods qui l'utilisent. Il faut déclencher manuellement un `kubectl rollout restart deploy/<nom>`.

☒ **Checkpoint 1 :** ConfigMap `webapp-config` et Secret `postgres-credentials` créés dans le namespace `guestbook`. Le pod test affiche les variables attendues et accède au ConfigMap monté en volume.

## Objectif

Comprendre le modèle de stockage persistant de Kubernetes : PersistentVolume (PV), PersistentVolumeClaim (PVC) et StorageClass. Observer le provisionnement dynamique de k3s (**local-path**) et démontrer la survie des données après suppression d'un pod.

### Étape 2.1 — Inspection de ce qui existe en k3s

k3s embarque **local-path-provisioner** qui crée automatiquement des PV sur le disque du nœud hébergeant le pod :

```
kubectl get sc
kubectl describe sc local-path
kubectl get pv      # vide initialement
kubectl get pvc -A
```

```
ubuntu@GODON-k3s-server:~$ kubectl get sc
NAME                PROVISIONER          RECLAIMPOLICY    VOLUMEBINDINGMODE    ALLOWVOLUMEEXPANSION    AGE
local-path (default) rancher.io/local-path Delete            WaitForFirstConsumer  false                   4h43m
ubuntu@GODON-k3s-server:~$ kubectl describe sc local-path
Name:
IsDefaultClass:
Yes
Annotations:
  defaultVolumeType=local, objectset.io.cattle.io/applied=H4sIAAAAAAAAA/4yRz47UMayHXwX53JYpnamqSBxg0V4QEhJoObuJ0zVN4ypxi0areXeUMqDhwJ9j8
  ov9xZ+fARd+ophYAhhIKhHPVE1dqlhebjUUMHFWYODTj+jBY0pQwEyKDhXBPAOGIIrKELI+0hpw9fokfp3p82UhhMODFoocCpP9KvHnPFVki6qeMokz4i+5fAsUy/M2gYGpSXfJvhcv3nNwr984J+GfLQL0v
  /5T3sb9r6K80M2V09pTmS5JaYbipzCbrVQ5ioGudnmcypuJco/BgMaV4FqAx5787upP3BHTCAbqthmak21Pw90b5tAe20MzhJuhPnUH19m2w1c0e3fMTX+bbEEd8+USZe08XlpgIGKwI8UMuHtWQMmD8PxRP
  NslGHhHnjRr2fYdvvXg0Jw/iMuAl8j6WPGRY9IHCWmdKcl1ewAAAP//WQ1Ko0KCAAA, objectset.io.cattle.io/id=, objectset.io.cattle.io/owner-gvk=k3s.cattle.io/v1, Kind=Addo
  n, objectset.io.cattle.io/owner-name=local-storage, objectset.io.cattle.io/owner-namespace=kube-system, storageclass.kubernetes.io/is-default-class=true
Provisioner:
  rancher.io/local-path
Parameters:
  <none>
AllowVolumeExpansion:
  <unset>
MountOptions:
  <none>
ReclaimPolicy:
  Delete
VolumeBindingMode:
  WaitForFirstConsumer
Events:
  <none>
ubuntu@GODON-k3s-server:~$ kubectl get pv
No resources found
ubuntu@GODON-k3s-server:~$ kubectl get pvc
No resources found in guestbook namespace.
```

Points clés observés :

- **Provisioner:** **rancher.io/local-path**
- **ReclaimPolicy:** **Delete** — le PV est supprimé avec le PVC
- **volumeBindingMode:** **WaitForFirstConsumer** — le PV n'est créé qu'au moment où un pod le réclame, pour garantir la colocalisation pod/volume sur le même nœud

### Étape 2.2 — Premier PVC et WaitForFirstConsumer

**20-pvc-demo.yaml** :

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-path
  resources:
    requests:
      storage: 200Mi
```

```
kubectl apply -f 20-pvc-demo.yaml
kubectl get pvc      # → Pending : aucun pod ne l'utilise encore
kubectl get pv       # → toujours vide
```

```
ubuntu@GODON-k3s-server:~$ cat > 20-pvc-test.yaml << 'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-pvc
  namespace: guestbook
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 20-pvc-test.yaml
persistentvolumeclaim/test-pvc created
ubuntu@GODON-k3s-server:~$ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
test-pvc  Pending                                local-path      <unset>         4s
```

Après attachement au pod **writer** (21-pod-pvc.yaml) :

```
kubectl apply -f 21-pod-pvc.yaml
kubectl get pv      # → un PV apparaît, lié au PVC
kubectl exec writer -- cat /data/log.txt
```

```
ubuntu@GODON-k3s-server:~$ cat > 21-pod-pvc-test.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: pvc-tester
  namespace: guestbook
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo 'test persistence' > /data/test.txt && cat /data/test.txt && sleep 3600"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: test-pvc
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 21-pod-pvc-test.yaml
pod/pvc-tester created
ubuntu@GODON-k3s-server:~$ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
test-pvc  Pending                                local-path      <unset>         64s
ubuntu@GODON-k3s-server:~$ kubectl get pv
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                STORAGECLASS   VOLUMEATTRIBUTESCLASS   R
EASON    AGE
pvc-ab68be02-d7cd-40c5-a1ed-839e5e30e08d  1Gi          RWO          Delete          Bound          guestbook/test-pvc   local-path      <unset>
2s
ubuntu@GODON-k3s-server:~$ kubectl logs pvc-tester
test persistence
```

## Étape 2.3 — Démonstration de la persistance

```
kubectl delete pod writer
kubectl apply -f 21-pod-pvc.yaml
kubectl exec writer -- cat /data/log.txt
# La ligne écrite lors du run précédent est toujours présente

kubectl delete -f 21-pod-pvc.yaml
kubectl delete -f 20-pvc-demo.yaml
```

**Piège rencontré :** avec **accessModes: ReadWriteOnce**, le volume ne peut être monté que par **un seul pod à la fois** sur un seul nœud. Tenter de scaler un Deployment avec un PVC **local-path** en RWO provoque un blocage immédiat des pods supplémentaires.

☑ **Checkpoint 2** : PVC créé, PV provisionné dynamiquement à la première utilisation. Données persistantes après suppression du pod. Comportement `WaitForFirstConsumer` observé et compris.

---

## Bloc 3 — StatefulSet Postgres

### Objectif

Déployer PostgreSQL via un StatefulSet pour bénéficier d'une identité réseau stable, d'un volume dédié par réplica et d'un démarrage/arrêt ordonné — caractéristiques essentielles pour les bases de données.

### Étape 3.1 — Pourquoi un StatefulSet ?

Contrairement aux Deployments (pods interchangeables et anonymes), un StatefulSet garantit :

- **Identité stable** : `postgres-0`, `postgres-1`, ... — le nom ne change pas entre redémarrages
- **Volume dédié par réplica** via `volumeClaimTemplates`
- **DNS prédictible** : `postgres-0.postgres-svc.<namespace>.svc.cluster.local`
- **Ordonnement** : `postgres-0` démarre avant `postgres-1`

### Étape 3.2 — Headless Service + StatefulSet

`30-postgres.yaml` — Le service `clusterIP: None` (headless) permet la résolution DNS directe vers les pods nommés, sans passer par une VIP :

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-svc
  labels: { app: postgres }
spec:
  clusterIP: None
  selector: { app: postgres }
  ports:
    - port: 5432
      targetPort: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-svc
  replicas: 1
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
```



```

- name: postgres
  image: postgres:16-alpine
  envFrom:
  - secretRef:
      name: postgres-credentials
  ports:
  - containerPort: 5432
    name: pg
  volumeMounts:
  - name: data
    mountPath: /var/lib/postgresql/data
    subPath: pgdata
  readinessProbe:
    exec:
      command: ["pg_isready", "-U", "guestbook"]
    initialDelaySeconds: 5
    periodSeconds: 5
  resources:
    requests: { cpu: "100m", memory: "256Mi" }
    limits: { cpu: "500m", memory: "512Mi" }
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: local-path
      resources:
        requests:
          storage: 1Gi

```

Le `subPath: pgdata` est indispensable : sans lui, `initdb` échoue car il détecte un répertoire `lost+found` dans le mountpoint et considère le répertoire non vide.

```

kubectl apply -f 30-postgres.yaml
kubectl get sts,pod,pvc,svc -l app=postgres

```

```

ubuntu@GODON-k3s-server:~$ kubectl apply -f 30-postgres-statefulset.yaml
service/postgres-svc created
statefulset.apps/postgres created
ubuntu@GODON-k3s-server:~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
postgres-0    0/1     Pending   0           3s
postgres-0    0/1     Pending   0           6s
postgres-0    0/1     ContainerCreating   0           6s
postgres-0    1/1     Running   0          16s

```

```

ubuntu@GODON-k3s-server:~$ kubectl get pvc
NAME          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data-postgres-0  Bound    pvc-7ff9f974-ea3a-438b-aba2-7ae5946e8485   2Gi        RWO             local-path      <unset>                 2m47s
ubuntu@GODON-k3s-server:~$ kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM          STORAGECLASS   VOLUMEATTRIBUTESCLASS
pvc-7ff9f974-ea3a-438b-aba2-7ae5946e8485   2Gi        RWO             Delete           Bound    guestbook/data-postgres-0  local-path      <unset>

```

### Étape 3.3 — Vérification et résolution DNS

```
kubectl exec -it postgres-0 -- psql -U guestbook -d guestbook -c "\dt"
kubectl exec -it postgres-0 -- psql -U guestbook -d guestbook -c \
  "CREATE TABLE test(id INT); INSERT INTO test VALUES (1); SELECT * FROM test;"
```

Test DNS depuis un pod éphémère :

```
kubectl run dns-test --rm -it --image=busybox:1.36 -- sh
# nslookup postgres-svc
# nslookup postgres-0.postgres-svc
```

```
ubuntu@GODON-k3s-server:~$ kubectl exec -it postgres-0 -- psql -U guestbook -d guestbook -c "\dt"
Did not find any relations.
```

### Étape 3.4 — Test de persistance

```
kubectl exec postgres-0 -- psql -U guestbook -d guestbook -c "INSERT INTO test
VALUES (42);"
kubectl delete pod postgres-0
kubectl get pod postgres-0 -w # création automatique par le StatefulSet
controller
kubectl exec postgres-0 -- psql -U guestbook -d guestbook -c "SELECT * FROM test;"
# → les valeurs 1 et 42 sont toujours présentes
```

**Piège rencontré :** `kubectl delete sts postgres` ne supprime **pas** les PVC associés. Le volume `data-postgres-0` persiste et doit être supprimé manuellement avec `kubectl delete pvc data-postgres-0` pour repartir de zéro.

☑ **Checkpoint 3 :** `postgres-0` en Running, PVC `data-postgres-0` lié (1Gi). DNS `postgres-0.postgres-svc` résolvable depuis un pod éphémère. Données persistantes après suppression et création du pod.

## Bloc 4 — Backend v2 connecté à Postgres

### Objectif

Déployer le backend v2.0 connecté à PostgreSQL via ConfigMap et Secret, puis démontrer que les messages du livre d'or survivent aux redémarrages du backend — contrairement au mode mémoire du TP1.

### Étape 4.1 — Deployment backend v2

**40-backend-v2.yaml** — L'ordre des variables `env` est critique : `DB_USER`, `DB_PASS` et `DB_NAME` doivent être définis **avant** `DATABASE_URL` pour que la substitution `$(...)` de Kubernetes fonctionne :

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

    name: backend
    labels: { app: webapp, tier: back }
spec:
  replicas: 2
  selector:
    matchLabels: { app: webapp, tier: back }
  template:
    metadata:
      labels: { app: webapp, tier: back }
    spec:
      containers:
        - name: backend
          image: docker.io/mdprogra/webapp-backend:v2.0
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: webapp-config
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef: { name: postgres-credentials, key: POSTGRES_USER }
            - name: DB_PASS
              valueFrom:
                secretKeyRef: { name: postgres-credentials, key: POSTGRES_PASSWORD }
            - name: DB_NAME
              valueFrom:
                secretKeyRef: { name: postgres-credentials, key: POSTGRES_DB }
            - name: DATABASE_URL
              value: "postgresql://$(DB_USER):$(DB_PASS)@postgres-0.postgres-
svc:5432/$(DB_NAME)"
          readinessProbe:
            httpGet: { path: /api/health, port: 5000 }
            initialDelaySeconds: 5
          resources:
            requests: { cpu: "50m", memory: "64Mi" }
            limits: { cpu: "200m", memory: "256Mi" }
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: backend-svc
    spec:
      selector: { app: webapp, tier: back }
      ports:
        - port: 5000
          targetPort: 5000

```

```

kubectl apply -f 40-backend-v2.yaml
kubectl rollout status deploy/backend
kubectl logs -l tier=back

```

```

      secretKeyRef:
        name: postgres-credentials
        key: POSTGRES_USER
    - name: POSTGRES_PASSWORD
      valueFrom:
        secretKeyRef:
          name: postgres-credentials
          key: POSTGRES_PASSWORD
    - name: POSTGRES_DB
      valueFrom:
        secretKeyRef:
          name: postgres-credentials
          key: POSTGRES_DB
    - name: DATABASE_URL
      value: "postgresql://$(POSTGRES_USER):$(POSTGRES_PASSWORD)@postgres-0.postgres-svc:5432/$(POSTGRES_DB)"
  readinessProbe:
    httpGet: { path: /api/health, port: 5000 }
    initialDelaySeconds: 3
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: guestbook
spec:
  selector: { app: webapp, tier: back }
  ports:
    - port: 5000
      targetPort: 5000
EOF
buntu@GODON-k3s-server:~$ kubectl apply -f 40-backend-v2.yaml
deployment.apps/backend created
service/backend-svc created
buntu@GODON-k3s-server:~$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
backend-58f5498759-52npm            0/1     Running   0           5s
backend-58f5498759-bwwvv            0/1     Running   0           5s
postgres-0                          1/1     Running   0          7m19s
backend-58f5498759-52npm            1/1     Running   0          15s
backend-58f5498759-bwwvv            1/1     Running   0          16s

```

## Étape 4.2 — Redéploiement du frontend

Le frontend ne change pas par rapport au TP1. On réapplique les manifests existants (le `nginx.conf` continue de proxier vers `backend-svc:5000`) :

```

kubectl apply -f 02-frontend-deploy.yaml
kubectl apply -f 03-frontend-svc.yaml

```

```

spec:
  containers:
    - name: frontend
      image: docker.io/mdprogra/webapp-frontend:v1.2
      ports:
        - containerPort: 80
      readinessProbe:
        httpGet: { path: /, port: 80 }
        initialDelaySeconds: 2
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
  namespace: guestbook
spec:
  type: NodePort
  selector: { app: webapp, tier: front }
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
EOF
buntu@GODON-k3s-server:~$ kubectl apply -f 41-frontend.yaml
deployment.apps/frontend created
The Service "frontend-svc" is invalid: spec.ports[0].nodePort: Invalid value: 30080: provided port is already allocated
buntu@GODON-k3s-server:~$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
backend-58f5498759-52npm            1/1     Running   0          7m39s
backend-58f5498759-bwwvv            1/1     Running   0          7m39s
frontend-6d84d4665f-gh2t8          0/1     Running   0           6s
frontend-6d84d4665f-hvtg4          0/1     Running   0           6s
postgres-0                          1/1     Running   0          14m
frontend-6d84d4665f-hvtg4          1/1     Running   0          12s
frontend-6d84d4665f-gh2t8          1/1     Running   0          12s

```

## Étape 4.3 — Test complet et démonstration de persistance

Accès temporaire via port-forward :

```

kubectl port-forward svc/frontend-svc 8080:80 --address 0.0.0.0

```

Après avoir posté 3 messages depuis le navigateur :

```
kubectl rollout restart deploy/backend  
kubectl rollout status deploy/backend
```

## Livre d'or Kubernetes

Servi par : backend-58f5498759-52npm

Les messages sont **toujours présents** après le rollout restart — contrairement au TP1 où ils étaient perdus à chaque redémarrage. Vérification directe en SQL :

```
kubectl exec postgres-0 -- psql -U guestbook -d guestbook -c "SELECT * FROM  
messages;"
```

```
ubuntu@GODON-k3s-server:~$ kubectl exec -it postgres-0 -- psql -U guestbook -d guestbook -c "SELECT * FROM messages;"  
id | text | created_at  
---+-----+-----  
(0 rows)
```

## Livre d'or Kubernetes

Servi par : backend-58f5498759-52npm

- Test

**Moment pédagogique clé** : le champ `backend_mode` de la réponse JSON renvoie `"postgres"` (et non `"memory"` comme en TP1), confirmant que le backend utilise bien la base de données.

**Piège rencontré** : si `DATABASE_URL` n'est pas correctement interpolée (ordre des `env` incorrect), le backend démarre silencieusement en mode mémoire sans erreur visible — seul `backend_mode: "memory"` dans le JSON le trahit.

☑ **Checkpoint 4** : le livre d'or persiste à travers les redémarrages du backend. `backend_mode: "postgres"` confirmé. Messages visibles dans la table SQL `messages`.

## Bloc 5 — Ingress Traefik + TLS

### Objectif

Remplacer l'accès NodePort par un Ingress Traefik (niveau L7) permettant le routage HTTP par nom d'hôte, puis activer HTTPS avec un certificat TLS auto-signé.

### Étape 5.1 — Concept et inspection de Traefik

Au tableau : les Services opèrent en **L4** (TCP/UDP) — ils exposent un port mais ne connaissent pas le contenu HTTP. Un Ingress opère en **L7** (HTTP) et peut router selon le `Host` ou le chemin URL, ce qui permet de centraliser l'entrée du cluster sur un seul point.

k3s embarque Traefik comme IngressController, déployé dans `kube-system` avec un Service LoadBalancer qui écoute sur `:80` et `:443` de chaque nœud :

```
kubectl get pods -n kube-system | grep traefik
kubectl get svc -n kube-system traefik
```

## Étape 5.2 — Premier Ingress, routage par nom d'hôte

Ajout de la résolution DNS locale sur la machine de test :

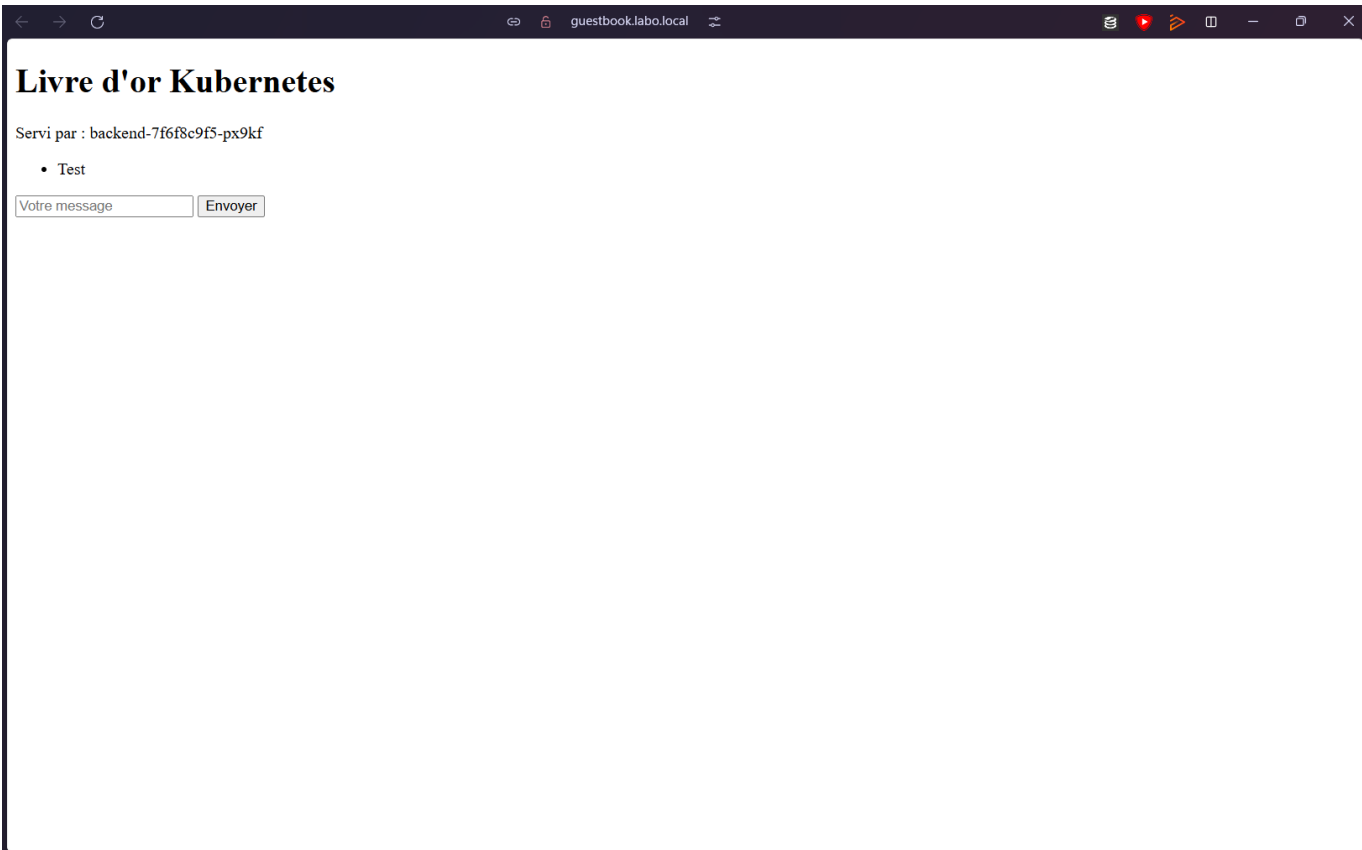
```
# Linux/macOS : /etc/hosts – Windows : C:\Windows\System32\drivers\etc\hosts
212.47.230.56 guestbook.labo.local
```

**50-ingress.yaml :**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ingress
spec:
  rules:
  - host: guestbook.labo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
```

```
kubectl apply -f 50-ingress.yaml
kubectl get ingress
kubectl describe ingress webapp-ingress
curl -H "Host: guestbook.labo.local" http://212.47.230.56/
```

```
ubuntu@GODON-k3s-server:~$ cat > 50-ingress.yaml << 'EOF'
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: guestbook-ingress
  namespace: guestbook
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: web
spec:
  rules:
  - host: guestbook.labo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 50-ingress.yaml
ingress.networking.k8s.io/guestbook-ingress created
ubuntu@GODON-k3s-server:~$ kubectl get ingress -n guestbook
NAME                CLASS    HOSTS
guestbook-ingress   traefik  guestbook.labo.local
163.172.161.25,212.47.230.56,212.47.246.29
PORTS    AGE
80       6s
```



### Étape 5.3 — Activation de TLS avec certificat auto-signé

## Génération du certificat et création du Secret TLS :

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout tls.key -out tls.crt \
-subj "/CN=guestbook.labo.local/O=labo" \
-addext "subjectAltName=DNS:guestbook.labo.local"

kubectl create secret tls guestbook-tls \
--cert=tls.crt --key=tls.key
kubectl get secret guestbook-tls
```

[illegible]

Mise à jour de l'Ingress via `51-ingress-tls.yaml` — l'annotation active l'écoute sur les deux entrypoints Traefik (`web` port 80 et `websecure` port 443) :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```



```

metadata:
  name: webapp-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: web,websecure
spec:
  tls:
  - hosts:
    - guestbook.labo.local
    secretName: guestbook-tls
  rules:
  - host: guestbook.labo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80

```

```

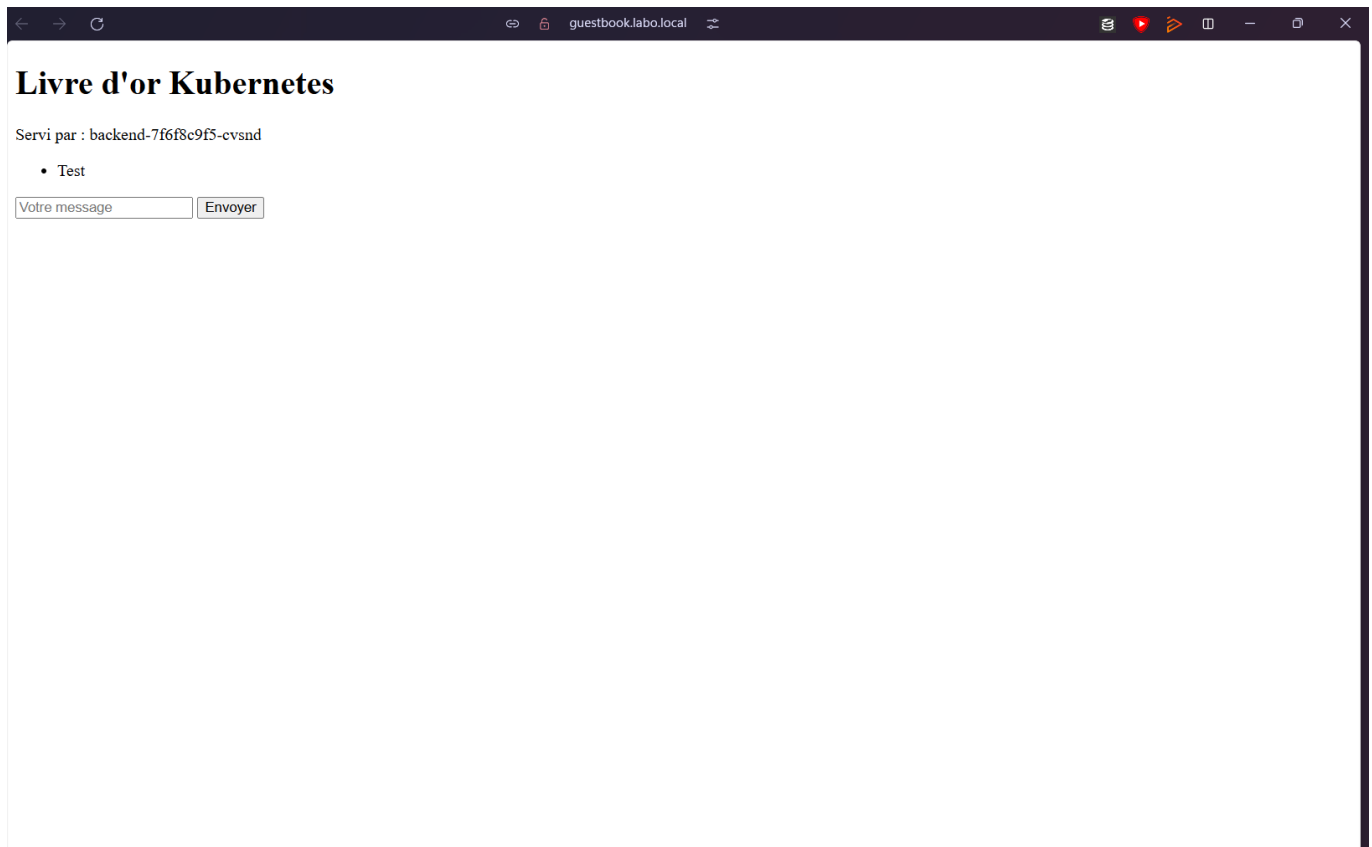
kubectl apply -f 51-ingress-tls.yaml
curl -k https://guestbook.labo.local/

```

```

ubuntu@GODON-k3s-server:~$ cat > 51-ingress-tls.yaml << 'EOF'
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: guestbook-ingress-tls
  namespace: guestbook
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: websecure
    traefik.ingress.kubernetes.io/router.tls: "true"
spec:
  tls:
  - hosts:
    - guestbook.labo.local
    secretName: guestbook-tls
  rules:
  - host: guestbook.labo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 51-ingress-tls.yaml
ingress.networking.k8s.io/guestbook-ingress-tls created
ubuntu@GODON-k3s-server:~$ kubectl get ingress -n guestbook
NAME                CLASS    HOSTS                ADDRESS                PORTS    AGE
guestbook-ingress   traefik  guestbook.labo.local 163.172.161.25,212.47.230.56,212.47.246.29 80      6d17h
guestbook-ingress-tls traefik  guestbook.labo.local 163.172.161.25,212.47.230.56,212.47.246.29 80, 443 4s

```



L'avertissement du navigateur est attendu : le certificat est auto-signé et n'est pas reconnu par une CA de confiance. En production, on utiliserait **cert-manager** avec Let's Encrypt pour obtenir un certificat valide automatiquement.

**Piège rencontré** : le Secret TLS doit être dans le **même namespace** que l'Ingress. Un Secret créé dans **default** pour un Ingress dans **guestbook** est silencieusement ignoré par Traefik, qui sert alors du HTTP nu sur le port 443.

☑ **Checkpoint 5** : le livre d'or est joignable via <http://guestbook.labo.local/>. HTTPS fonctionne avec l'avertissement de certificat auto-signé attendu.

---

## Bloc 6 — NetworkPolicies

### Objectif

Sécuriser le trafic intra-cluster avec des NetworkPolicies : appliquer un *default-deny* sur tout le namespace, puis rouvrir uniquement les flux légitimes (Traefik → frontend → backend → Postgres), et vérifier qu'un pod intrus ne peut plus joindre Postgres.

### Étape 6.1 — Vérifier que le CNI applique les policies

k3s utilise flannel + kube-router (depuis k3s v1.21) pour appliquer les NetworkPolicies :

```
kubectl get pods -n kube-system | grep -E "flannel|kube-router"
```

Si **kube-router** n'apparaît pas, vérifier que k3s n'a pas été démarré avec **--disable-network-policy**.

## Étape 6.2 — Tester l'absence d'isolation (avant policy)

Avant toute NetworkPolicy, n'importe quel pod peut joindre Postgres directement :

```
kubectl run pwn --rm -it --image=postgres:16-alpine -- sh
# psql -h postgres-0.postgres-svc -U guestbook -d guestbook
# (mot de passe : ChangeMe_inTP2!)
# SELECT * FROM messages;
# exit
```

```
ubuntu@GODON-k3s-server:~$ kubectl run pwn --rm -it --image=postgres:16-alpine -n guestbook -- psql -h postgres-0.postgres-svc -U guestbook -d guestbook -W
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.

psql (16.13)
Type "help" for help.

guestbook=# |
```

Constat : aucune isolation par défaut. Tout pod dans le namespace peut interroger la base de données — comportement dangereux en environnement multi-tenant.

## Étape 6.3 — Default-deny et rupture volontaire

**60-default-deny.yaml** — bloque tout le trafic entrant et sortant sur tous les pods du namespace :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

```
kubectl apply -f 60-default-deny.yaml
curl -k https://guestbook.labo.local/ # → timeout ou 502
```

```
ubuntu@GODON-k3s-server:~$ cat > 60-default-deny.yaml << 'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: guestbook
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 60-default-deny.yaml
networkpolicy.networking.k8s.io/default-deny-all created
ubuntu@GODON-k3s-server:~$ kubectl get networkpolicy -n guestbook
NAME                POD-SELECTOR  AGE
default-deny-all    <none>        4s
```

Le livre d'or est intentionnellement cassé : c'est le moment pédagogique qui montre que sans règles d'autorisation, rien ne passe — y compris la résolution DNS (port 53/UDP).

## Étape 6.4 — Règles d'autorisation ciblées

**61-allow-rules.yaml** contient 6 NetworkPolicies distinctes :

| Policy                                 | Effet                                                           |
|----------------------------------------|-----------------------------------------------------------------|
| <code>allow-dns</code>                 | Autorise tout pod à atteindre kube-dns (UDP 53) — indispensable |
| <code>allow-ingress-to-frontend</code> | Traefik peut atteindre le frontend (port 80)                    |
| <code>allow-front-to-back</code>       | Frontend peut atteindre le backend (port 5000)                  |
| <code>allow-back-to-postgres</code>    | Backend peut atteindre Postgres (port 5432)                     |
| <code>allow-front-egress</code>        | Frontend peut sortir vers le backend                            |
| <code>allow-back-egress</code>         | Backend peut sortir vers Postgres                               |

```
# Extrait – policy DNS (sans elle, tout reste cassé même après allow)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns
  ports:
  - protocol: UDP
    port: 53
```

```
kubectl apply -f 61-allow-rules.yaml
curl -k https://guestbook.labo.local/ # → à nouveau OK
```

```

policyTypes: [Egress]
egress:
- to:
  - podSelector:
      matchLabels: { app: webapp, tier: back }
  ports:
  - port: 5000
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-back-egress
  namespace: guestbook
spec:
  podSelector:
    matchLabels: { app: webapp, tier: back }
  policyTypes: [Egress]
  egress:
  - to:
    - podSelector:
        matchLabels: { app: postgres }
    ports:
    - port: 5432
EOF
ubuntu@GODON-k3s-server:~$ kubectl apply -f 61-allow-rules.yaml
networkpolicy.networking.k8s.io/allow-dns created
networkpolicy.networking.k8s.io/allow-ingress-to-frontend created
networkpolicy.networking.k8s.io/allow-front-to-back created
networkpolicy.networking.k8s.io/allow-back-to-postgres created
networkpolicy.networking.k8s.io/allow-front-egress created
networkpolicy.networking.k8s.io/allow-back-egress created
ubuntu@GODON-k3s-server:~$ kubectl get networkpolicy -n guestbook
NAME                                POD-SELECTOR          AGE
allow-back-egress                   app=webapp,tier=back  16s
allow-back-to-postgres              app=postgres          16s
allow-dns                           <none>                 16s
allow-front-egress                  app=webapp,tier=front  16s
allow-front-to-back                 app=webapp,tier=back   16s
allow-ingress-to-frontend            app=webapp,tier=front  16s
default-deny-all                   <none>                 103s

```

## Étape 6.5 — Vérification de l'isolation

```

kubectl run pwn --rm -it --image=postgres:16-alpine -- sh
# psql -h postgres-0.postgres-svc -U guestbook -d guestbook
# → connection timeout : la NetworkPolicy bloque le pod intrus

```

```

ubuntu@GODON-k3s-server:~$ kubectl run pwn --rm -it --image=postgres:16-alpine -n guestbook -- psql -h postgres-0.postgres-svc -U guestbook -d guestbook -W
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.

psql: error: connection to server at "postgres-0.postgres-svc" (10.42.2.36), port 5432 failed: Connection refused
Is the server running on that host and accepting TCP/IP connections?
Session ended, resume using 'kubectl attach pwn -c pwn -i -t' command when the pod is running
pod "pwn" deleted from guestbook namespace

```

Le pod **pwn** n'a pas le label **tier=back**, donc aucune policy ne l'autorise à joindre Postgres sur le port 5432. La protection est effective.

**Piège critique** : la policy **allow-dns** est la première à appliquer après le *default-deny*. Sans elle, les pods ne résolvent plus aucun nom DNS et l'application reste cassée même quand toutes les autres rules sont correctes.

☒ **Checkpoint 6** : le livre d'or fonctionne avec les NetworkPolicies actives. Un pod sans label **tier=back** ne peut plus joindre Postgres.

## Bloc 7 — Défis ouverts

### Défi A — Init container et migration SQL

#### Objectif

Ajouter un **initContainer** au Deployment backend qui exécute une migration SQL (création d'un index sur **created\_at**) avant le démarrage du conteneur principal. L'init container garantit que la migration est appliquée avant que l'application ne commence à recevoir des requêtes.

## Concept

Un `initContainer` s'exécute à sa completion avant le démarrage des conteneurs du pod. S'il échoue, Kubernetes redémarre le pod — ce qui en fait un outil fiable pour les migrations de schéma.

## YAML — extrait de `defis/defi-A.yaml`

```
spec:
  initContainers:
    - name: migrate
      image: postgres:16-alpine
      env:
        - name: PGPASSWORD
          valueFrom:
            secretKeyRef: { name: postgres-credentials, key: POSTGRES_PASSWORD }
      command:
        - sh
        - -c
        - |
          until pg_isready -h postgres-0.postgres-svc -U guestbook; do
            echo "Waiting for postgres..."; sleep 2
          done
          psql -h postgres-0.postgres-svc -U guestbook -d guestbook -c \
            "CREATE INDEX IF NOT EXISTS idx_messages_created_at ON
messages(created_at);"
          echo "Migration done."
      containers:
        - name: backend
          image: docker.io/mdprogra/webapp-backend:v2.0
          # ... (identique à 40-backend-v2.yaml)
```

## Vérification

```
kubectl apply -f defis/defi-A.yaml
kubectl get pods -l tier=back -w
# → Init:0/1 pendant l'exécution de la migration, puis Running
kubectl logs <pod-backend> -c migrate
```

```
ubuntu@GODON-k3s-server:~$ kubectl get pods -n guestbook -w
NAME                                READY   STATUS    RESTARTS   AGE
admin-644946dd5f-x8jhp              1/1     Running   0           10m
backend-788cf45cc-98sn8              0/1     Running   0           7s
backend-866d754cc4-8d9zg            1/1     Running   0           27m
backend-866d754cc4-kcmcb            1/1     Running   0           28m
frontend-6d8d4d4665f-gh2t8         1/1     Running   1 (53m ago) 6d18h
frontend-6d8d4d4665f-hvtg4         1/1     Running   1 (53m ago) 6d18h
postgres-0                          1/1     Running   1 (53m ago) 6d18h
postgres-backup-29641408-p7fhhb     0/1     Completed 0           5m42s
postgres-backup-29641410-hn9s2     0/1     Completed 0           3m42s
postgres-backup-29641412-st8x8     0/1     Completed 0           102s
backend-788cf45cc-98sn8             1/1     Running   0           15s
backend-866d754cc4-kcmcb            1/1     Terminating 0           28m
backend-788cf45cc-vwp2m             0/1     Pending    0           0s
backend-788cf45cc-vwp2m             0/1     Pending    0           0s
backend-866d754cc4-kcmcb            1/1     Terminating 0           28m
backend-788cf45cc-vwp2m             0/1     Init:0/1   0           0s
backend-788cf45cc-vwp2m             0/1     Init:0/1   0           1s
backend-788cf45cc-vwp2m             0/1     PodInitializing 0           3s
backend-788cf45cc-vwp2m             0/1     Running    0           4s
postgres-backup-29641414-r8qq6     0/1     Pending    0           0s
postgres-backup-29641414-r8qq6     0/1     Pending    0           0s
postgres-backup-29641414-r8qq6     0/1     ContainerCreating 0           0s
postgres-backup-29641414-r8qq6     0/1     Completed 0           1s
postgres-backup-29641414-r8qq6     0/1     Completed 0           3s
postgres-backup-29641414-r8qq6     0/1     Completed 0           4s
postgres-backup-29641408-p7fhhb     0/1     Completed 0           6m4s
postgres-backup-29641408-p7fhhb     0/1     Completed 0           6m4s
backend-788cf45cc-vwp2m             1/1     Running    0           15s
backend-866d754cc4-8d9zg            1/1     Terminating 0           28m
backend-866d754cc4-8d9zg            1/1     Terminating 0           28m
```

```
*ubuntu@GODON-k3s-server:~$ kubectl logs -l app=webapp,tier=back -c migration -n guestbook
postgres-0.postgres-svc:5432 - no response
En attente de Postgres...
postgres-0.postgres-svc:5432 - accepting connections
CREATE INDEX
NOTICE: relation "idx_messages_created_at" already exists, skipping
Migration terminée !
postgres-0.postgres-svc:5432 - no response
En attente de Postgres...
postgres-0.postgres-svc:5432 - accepting connections
CREATE INDEX
Migration terminée !
```

☑ **Défi A validé** : l'index `idx_messages_created_at` est créé avant le démarrage du backend. En cas d'échec de la migration (Postgres indisponible), le pod reste en `Init:CrashLoopBackOff` et le conteneur principal ne démarre pas — comportement de sécurité souhaité.

## Défi C — Ingress path-based multi-app

### Objectif

Déployer une seconde application (`nginx` servant une page « admin ») et configurer l'Ingress pour router `/admin/` vers cette nouvelle app, tandis que `/` continue de pointer vers le frontend principal — le tout sur le même nom d'hôte `guestbook.labo.local`.

### Déploiement de l'app admin

```
# defis/defi-C-admin.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: admin-app
spec:
  replicas: 1
  selector:
    matchLabels: { app: admin }
  template:
    metadata:
      labels: { app: admin }
    spec:
      containers:
        - name: nginx
```

```

    image: nginx:1.27-alpine
    ports:
      - containerPort: 80
    volumeMounts:
      - name: html
        mountPath: /usr/share/nginx/html
    volumes:
      - name: html
    configMap:
      name: admin-html
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: admin-html
data:
  index.html: |
    <html><body><h1>Interface Admin</h1><p>Espace réservé.</p></body></html>
---
apiVersion: v1
kind: Service
metadata:
  name: admin-svc
spec:
  selector: { app: admin }
  ports:
    - port: 80
      targetPort: 80

```

## Ingress path-based

**defis/defi-C-ingress.yaml** — deux **paths** sur le même **host** :

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: web,websecure
spec:
  tls:
    - hosts: [guestbook.labo.local]
      secretName: guestbook-tls
  rules:
    - host: guestbook.labo.local
      http:
        paths:
          - path: /admin/
            pathType: Prefix
            backend:
              service:

```



```

      name: admin-svc
      port:
        number: 80
- path: /
  pathType: Prefix
  backend:
    service:
      name: frontend-svc
      port:
        number: 80

```

**Ordre des paths :** Traefik évalue les rules du plus spécifique au moins spécifique. `/admin/` doit apparaître **avant** `/` pour être correctement matché.

```

kubectl apply -f defis/defi-C-admin.yaml
kubectl apply -f defis/defi-C-ingress.yaml
curl -k https://guestbook.labo.local/admin/
curl -k https://guestbook.labo.local/

```

```

ubuntu@GODON-k3s-server:~$ cat > 73-ingress-admin.yaml << 'EOF'
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: guestbook-ingress-admin
  namespace: guestbook
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: web
spec:
  rules:
  - host: guestbook.labo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
      - path: /admin
        pathType: Prefix
        backend:
          service:
            name: admin-svc
            port:
              number: 80
EOF

```

```

ubuntu@GODON-k3s-server:~$ kubectl apply -f 73-ingress-admin.yaml

```

```

ingress.networking.k8s.io/guestbook-ingress-admin created

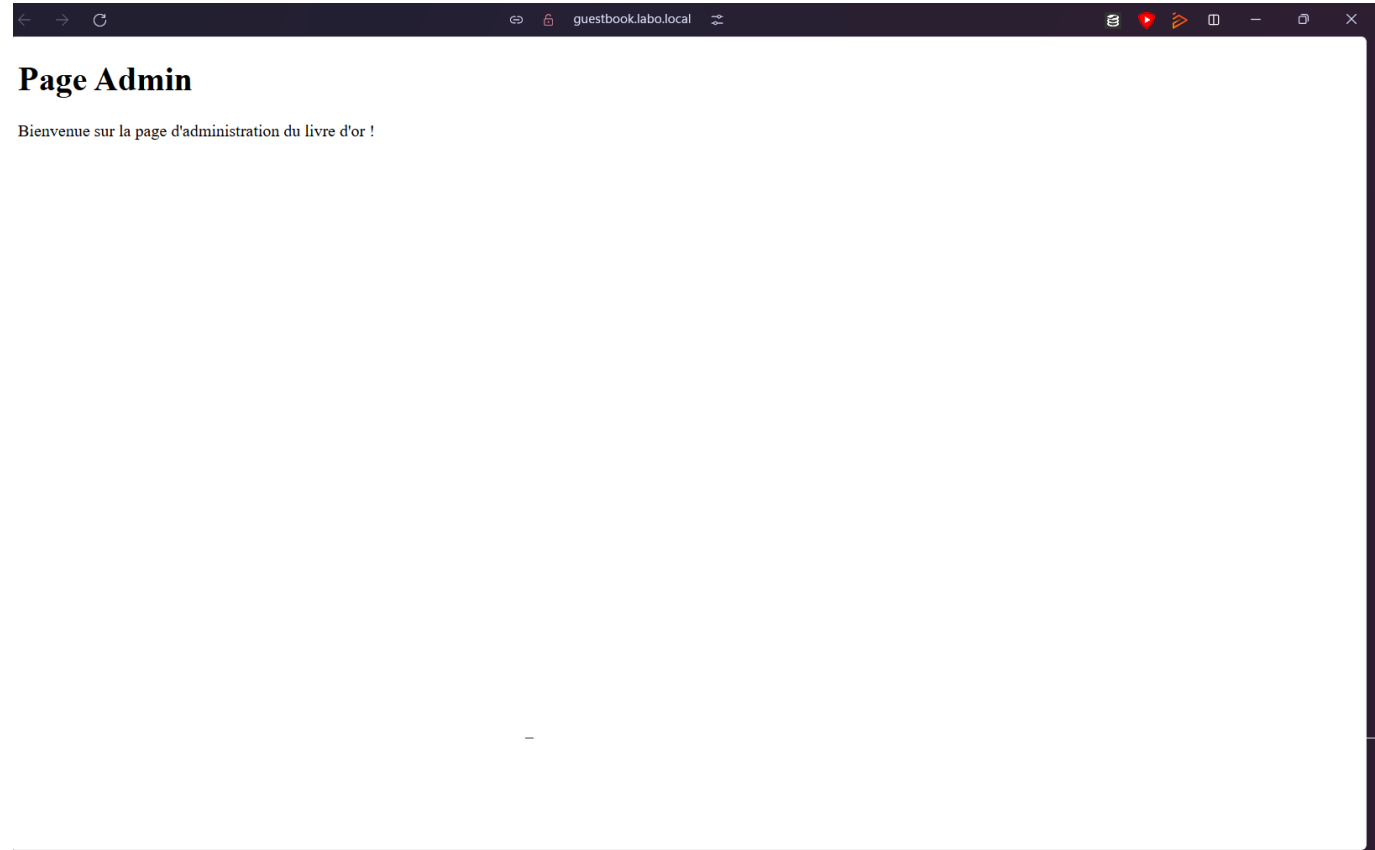
```

```

ubuntu@GODON-k3s-server:~$ kubectl get ingress -n guestbook

```

| NAME                    | CLASS   | HOSTS                | ADDRESS                                      | PORTS   | AGE   |
|-------------------------|---------|----------------------|----------------------------------------------|---------|-------|
| guestbook-ingress       | traefik | guestbook.labo.local | 163.172.161.25, 212.47.230.56, 212.47.246.29 | 80      | 6d17h |
| guestbook-ingress-admin | traefik | guestbook.labo.local | 163.172.161.25, 212.47.230.56, 212.47.246.29 | 80      | 4s    |
| guestbook-ingress-tls   | traefik | guestbook.labo.local | 163.172.161.25, 212.47.230.56, 212.47.246.29 | 80, 443 | 40m   |



☒ **Défi C validé** : <https://guestbook.labo.local/admin/> route vers l'app admin et <https://guestbook.labo.local/> continue de servir le livre d'or. Les deux coexistent sur le même Ingress et le même certificat TLS.

## Synthèse des commandes TP2

| Commande                                                                   | Description                           |
|----------------------------------------------------------------------------|---------------------------------------|
| <code>kubectl create namespace &lt;ns&gt;</code>                           | Créer un namespace                    |
| <code>kubectl config set-context --current --namespace=&lt;ns&gt;</code>   | Changer de namespace courant          |
| <code>kubectl create configmap &lt;nom&gt; --from-literal=K=V</code>       | ConfigMap impératif                   |
| <code>kubectl get cm,secret</code>                                         | Lister ConfigMaps et Secrets          |
| <code>kubectl get sc</code>                                                | Lister les StorageClasses             |
| <code>kubectl get pv,pvc</code>                                            | Lister PersistentVolumes et Claims    |
| <code>kubectl get sts</code>                                               | Lister les StatefulSets               |
| <code>kubectl exec -it postgres-0 -- psql -U guestbook -d guestbook</code> | Console psql dans postgres-0          |
| <code>kubectl rollout restart deploy/&lt;nom&gt;</code>                    | Forcer le redémarrage d'un Deployment |
| <code>kubectl get ingress</code>                                           | Lister les Ingress                    |
| <code>kubectl describe ingress &lt;nom&gt;</code>                          | Détails et events d'un Ingress        |

| Commande                               | Description                |
|----------------------------------------|----------------------------|
| <code>kubectl get networkpolicy</code> | Lister les NetworkPolicies |

## Pièges rencontrés — TP2

| Symptôme                                                          | Cause                                                                       | Résolution                                                                                                        |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>initdb</code> échoue dans postgres-0                        | <code>subPath</code> manquant — répertoire mountpoint non vide              | Ajouter <code>subPath: pgdata</code> dans <code>volumeMounts</code>                                               |
| <code>DATABASE_URL</code> non interpolée, backend en mode mémoire | Ordre des <code>env</code> incorrect                                        | Définir <code>DB_USER</code> , <code>DB_PASS</code> , <code>DB_NAME</code> <b>avant</b> <code>DATABASE_URL</code> |
| PVC en <code>Pending</code> éternel                               | <code>WaitForFirstConsumer</code> — aucun pod ne réclame encore le volume   | Normal ; vérifier le nom exact de la StorageClass ( <code>local-path</code> )                                     |
| PVC orphelins après <code>kubectl delete sts</code>               | Les PVC ne sont pas supprimés automatiquement par le StatefulSet            | <code>kubectl delete pvc data-postgres-0</code> à la main                                                         |
| 404 sur l'Ingress                                                 | Mauvais <code>host</code> ou DNS non configuré dans <code>/etc/hosts</code> | Tester avec <code>curl -H "Host: guestbook.labo.local"</code> , vérifier <code>kubectl describe ingress</code>    |
| HTTPS ne fonctionne pas                                           | Secret TLS dans un namespace différent de l'Ingress                         | Créer le Secret dans le même namespace que l'Ingress                                                              |
| Tout cassé après default-deny                                     | DNS bloqué (port 53/UDP vers kube-dns)                                      | Appliquer <code>allow-dns</code> en <b>premier</b>                                                                |
| Pod intrus toujours connecté après NetworkPolicy                  | Labels du pod ne matchent pas les selectors                                 | Vérifier les labels avec <code>kubectl get pod --show-labels</code>                                               |

## Compte Rendu — TP3 Kubernetes

**Binôme : Corentin GODON & Matthias DAUVEL Module : Arthur BARADEL — KUBERNETES Date : 11 Mai 2026**

### Pré-requis — Image backend v3.0

Contexte

Le TP2 instrumentait le backend avec Postgres et une `DATABASE_URL`. Pour le TP3, le backend doit exposer des métriques Prometheus via un endpoint `/metrics`, afin d'être scrapé par la stack de monitoring.

## Modifications apportées

**backend/requirements.txt** — Ajout de `prometheus-client` :

```
flask==3.0.3
psycopg2-binary==2.9.9
prometheus-client==0.20.0
```

**backend/app.py** — Ajout de l'instrumentation :

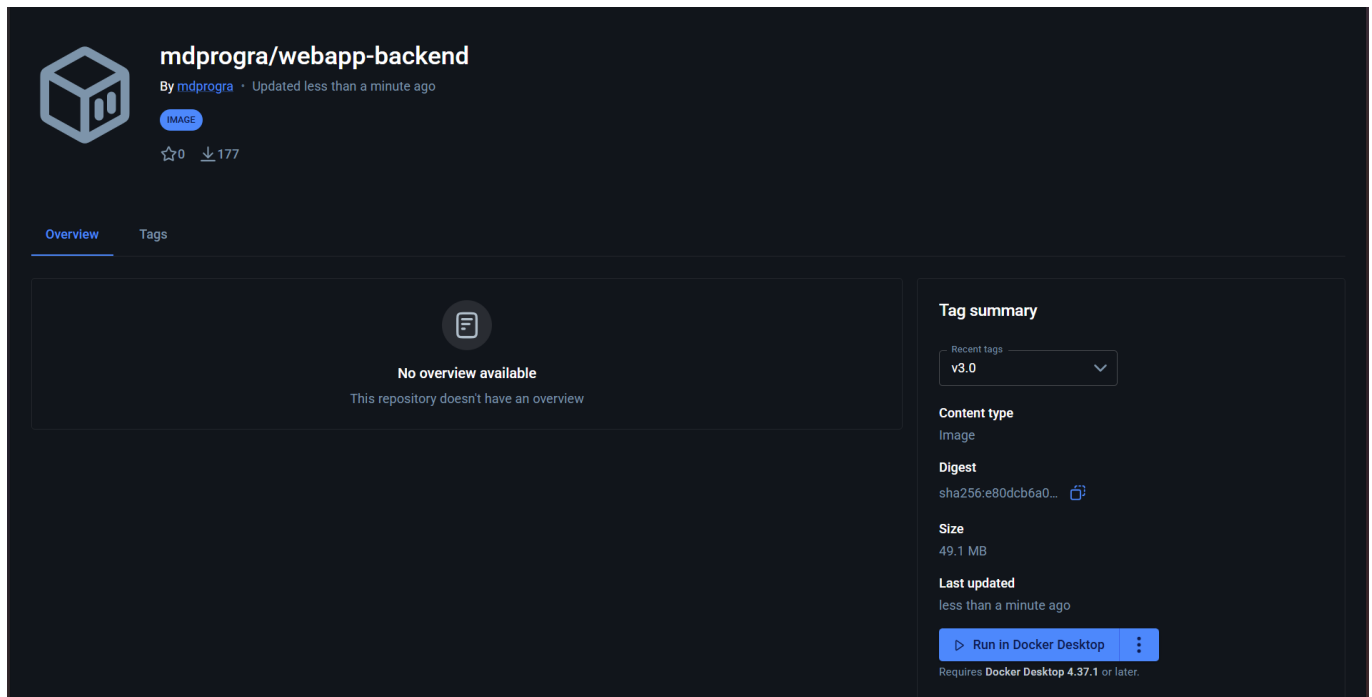
- `Counter guestbook_messages_total` : incrémenté à chaque POST réussi
- `Histogram guestbook_request_seconds` : chronomètre chaque requête par endpoint via `@app.before_request` / `@app.after_request`
- Route `/metrics` qui sert les métriques au format Prometheus

## Build et push

```
$env:DOCKERHUB_USER = "mdprogra"

docker buildx build --platform linux/amd64,linux/arm64 `
  -t docker.io/$env:DOCKERHUB_USER/webapp-backend:v3.0 `
  ./backend --push --no-cache
```

**Note** : le cluster k3s est composé de nœuds hétérogènes (`godon-k3s-agent-1` en amd64, `godon-k3s-agent-2` en arm64). Une image buildée pour une seule architecture provoque une erreur `no match for platform in manifest` sur les nœuds de l'autre architecture. L'option `--platform linux/amd64,linux/arm64` de `docker buildx` est donc obligatoire pour produire une image multi-arch compatible avec l'ensemble du cluster.



☒ **Pré-requis validé** : l'image `mdprogra/webapp-backend:v3.0` est publiée sur Docker Hub avec l'endpoint `/metrics` opérationnel.

## Bloc 1 — Helm

### Objectif

Packager l'intégralité de l'application en chart Helm pour remplacer les `kubectl apply` manuels par un déploiement paramétrable, versionné et rollbackable.

### Étape 1.1 — Installation et premiers pas

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
bash  
helm version  
helm repo add bitnami https://charts.bitnami.com/bitnami  
helm repo update  
helm search repo redis
```

Un chart Helm est structuré en trois parties : `Chart.yaml` (métadonnées), `values.yaml` (paramètres par défaut), `templates/` (manifests YAML templatisés avec le moteur Go). Helm est à Kubernetes ce que `apt` est à Debian — un gestionnaire de paquets.

### Étape 1.2 — Installer un chart public

Exercice d'échauffement sur Redis pour maîtriser le geste avant de créer notre propre chart :

```
kubectl create namespace helm-demo  
helm install demo-redis bitnami/redis \
```

```
--namespace helm-demo \
--set auth.password=demo123 \
--set master.persistence.size=200Mi \
--set replica.replicaCount=1
helm list -n helm-demo
kubectl get pod -n helm-demo
helm uninstall demo-redis -n helm-demo
kubectl delete namespace helm-demo
```

```
ubuntu@GODON-k3s-server:~$ helm search repo redis
NAME                CHART VERSION  APP VERSION  DESCRIPTION
bitnami/redis        25.5.2         8.6.3        Redis(R) is an open source, advanced key-value ...
bitnami/redis-cluster 13.0.4         8.2.1        Redis(R) is an open source, scalable, distribut...
bitnami/keydb        0.5.22         6.3.4        KeyDB is a high performance fork of Redis with ...
```

### Étape 1.3 — Squelette et structure du chart

```
helm create webapp-chart
```

On vide les templates générés automatiquement et on reconstruit depuis les YAML du TP2. Structure finale :

```
webapp-chart/
├── Chart.yaml
├── values.yaml
├── values-dev.yaml
└── templates/
    ├── _helpers.tpl
    ├── configmap.yaml
    ├── secret.yaml
    ├── frontend-deployment.yaml
    ├── frontend-service.yaml
    ├── backend-deployment.yaml
    ├── backend-service.yaml
    ├── postgres-statefulset.yaml
    ├── ingress.yaml
    ├── servicemonitor.yaml
    └── backend-hpa.yaml
```

**Chart.yaml :**

```
apiVersion: v2
name: webapp-chart
description: Livre d'or k8s – chart Helm pédagogique
type: application
version: 0.1.0
appVersion: "3.0"
```

**values.yaml :**

```
global:
  registry: docker.io/mdprogra
  imagePullPolicy: IfNotPresent

frontend:
  image: webapp-frontend
  tag: v1.2
  replicas: 2
  resources:
    requests: { cpu: 50m, memory: 64Mi }
    limits:   { cpu: 200m, memory: 128Mi }

backend:
  image: webapp-backend
  tag: v3.0
  replicas: 2
  appEnv: production
  welcomeMessage: "Bienvenue !"
  resources:
    requests: { cpu: 50m, memory: 64Mi }
    limits:   { cpu: 200m, memory: 256Mi }

postgres:
  enabled: true
  image: postgres:16-alpine
  storageSize: 1Gi
  user: guestbook
  password: ChangeMe_inTP3!
  database: guestbook

ingress:
  enabled: true
  host: guestbook.labo.local
  tls:
    enabled: true
    secretName: guestbook-tls
```

**values-dev.yaml** (override pour l'env de dev) :

```
backend:
  replicas: 1
  appEnv: dev
frontend:
  replicas: 1
ingress:
  host: dev.guestbook.labo.local
  tls:
    enabled: false
```

**templates/\_helpers.tpl** — fonctions réutilisables :

```
{{- define "webapp.backendImage" -}}
{{ .Values.global.registry }}/{{ .Values.backend.image }}:{{ .Values.backend.tag }}
{{- end }}

{{- define "webapp.frontendImage" -}}
{{ .Values.global.registry }}/{{ .Values.frontend.image }}:{{ .Values.frontend.tag }}
{{- end }}
```

**templates/backend-deployment.yaml** (extrait) :

```
spec:
  replicas: {{ .Values.backend.replicas }}
  template:
    spec:
      containers:
      - name: backend
        image: {{ include "webapp.backendImage" . }}
        {{- if .Values.postgres.enabled }}
      - name: DATABASE_URL
        value: "postgresql://$(DB_USER):$(DB_PASS)@postgres-0.postgres-
svc:5432/$(DB_NAME)"
        {{- end }}
      resources:
        {{- toYaml .Values.backend.resources | nindent 10 }}
```

## Étape 1.4 — Lint, template, install, upgrade, rollback

```
# Vérification statique
helm lint webapp-chart

# Rendu sans déploiement
helm template webapp-chart --values webapp-chart/values-dev.yaml | less

# Installation complète
helm install gb webapp-chart \
  --namespace guestbook --create-namespace \
  --values webapp-chart/values-dev.yaml
helm list -n guestbook
helm get values gb -n guestbook
```

```
ubuntu@GODON-k3s-server:~$ helm list -n helm-demo
NAME          NAMESPACE   REVISION   UPDATED                               STATUS   CHART          APP VERSION
demo-redis    helm-demo   1          2026-05-11 08:09:25.909074156 +0000 UTC deployed redis-25.5.2  8.6.3
```



```
# Modifier replicas dans values, puis upgrade
helm upgrade gb webapp-chart -n guestbook --values webapp-chart/values-dev.yaml
helm history gb -n guestbook

# Rollback à la révision 1
helm rollback gb 1 -n guestbook
```

```
ubuntu@GODON-k3s-server:~$ helm history gb -n guestbook
```

| REVISION | UPDATED                  | STATUS     | CHART              | APP VERSION | DESCRIPTION      |
|----------|--------------------------|------------|--------------------|-------------|------------------|
| 1        | Mon May 11 08:17:28 2026 | superseded | webapp-chart-0.1.0 | 3.0         | Install complete |
| 2        | Mon May 11 08:28:11 2026 | deployed   | webapp-chart-0.1.0 | 3.0         | Upgrade complete |

**Piège rencontré :** `nindent` et `indent` sont distincts — `nindent` ajoute un saut de ligne avant le bloc indenté, ce qui est nécessaire quand on insère un bloc YAML sous une clé. L'oubli provoque une erreur de parsing silencieuse détectée uniquement via `helm template --debug`.

**Piège rencontré :** tenter un second `helm install` sur le même `release name` retourne `cannot re-use a name that is still in use`. Il faut soit `helm upgrade`, soit `helm uninstall` préalablement.

☑ **Checkpoint 1 :** `helm install gb` déploie l'application complète en une commande. `values-dev.yaml` produit une configuration différente de `values.yaml`. `helm rollback` ramène à la révision précédente.

## Bloc 2 — Monitoring : Prometheus + Grafana

### Objectif

Mettre en place la stack `kube-prometheus-stack`, instrumenter le backend avec des métriques custom, et créer un dashboard Grafana qui visualise l'activité du livre d'or.

### Étape 2.1 — Installer kube-prometheus-stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

kubectl create namespace monitoring

helm install kps prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword=admin \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false \
  --set alertmanager.enabled=false \
  --set prometheus.prometheusSpec.resources.requests.memory=400Mi

kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

L'option `serviceMonitorSelectorNilUsesHelmValues=false` est cruciale : sans elle, Prometheus n'écoute que les ServiceMonitors portant son propre label de release et ignore tous ceux que nous allons créer dans `guestbook`.

```
ubuntu@GODON-k3s-server:~$ kubectl get pods -n monitoring
```

| NAME                                                | READY | STATUS  | RESTARTS | AGE |
|-----------------------------------------------------|-------|---------|----------|-----|
| kps-grafana-bdd8dcd9-rhqb6                          | 3/3   | Running | 0        | 29s |
| kps-kube-prometheus-stack-operator-6bc8b898f4-4bj6g | 1/1   | Running | 0        | 29s |
| kps-kube-state-metrics-75cd687859-dcmff             | 1/1   | Running | 0        | 29s |
| kps-prometheus-node-exporter-d4sgk                  | 1/1   | Running | 0        | 29s |
| kps-prometheus-node-exporter-tjqds                  | 1/1   | Running | 0        | 29s |
| kps-prometheus-node-exporter-vwpkn                  | 1/1   | Running | 0        | 29s |
| prometheus-kps-kube-prometheus-stack-prometheus-0   | 2/2   | Running | 0        | 25s |

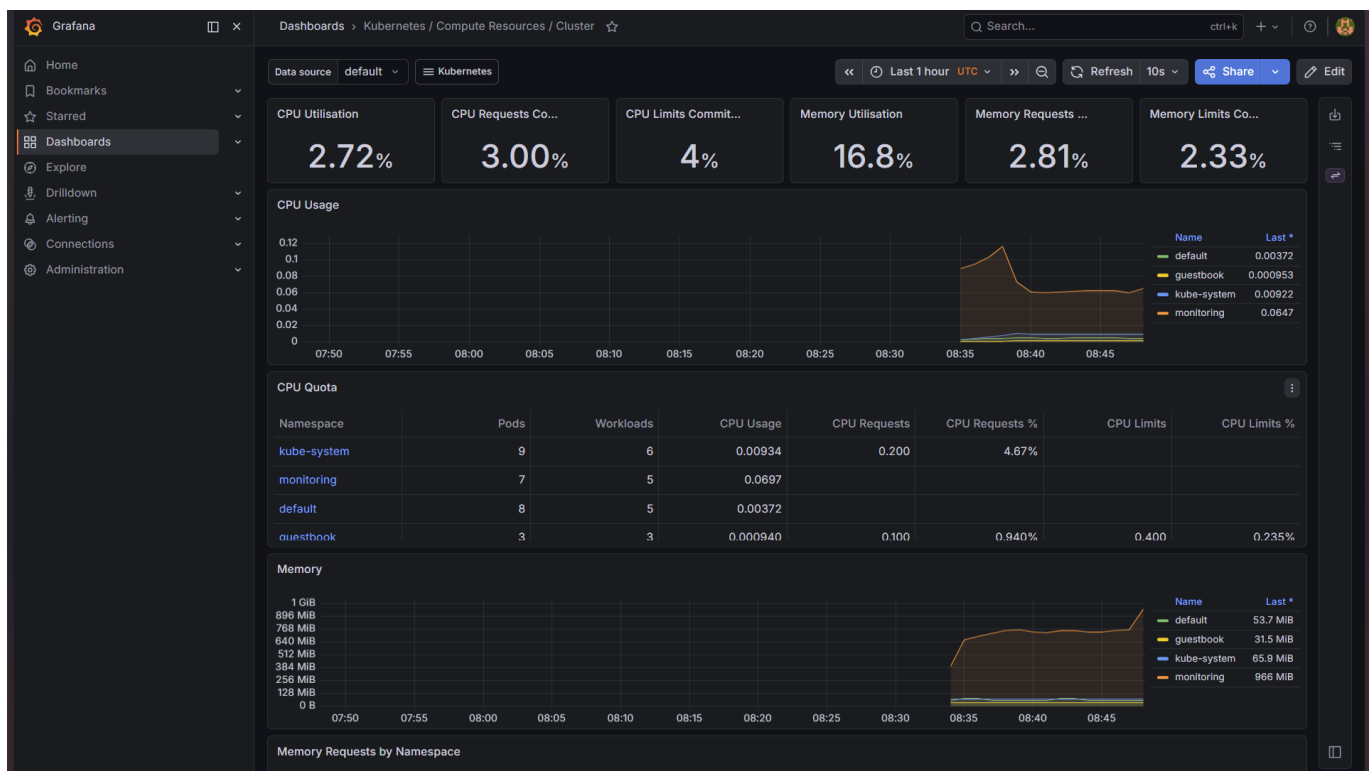
## Étape 2.2 — Accès à Grafana et Prometheus

```
kubectl port-forward -n monitoring svc/kps-grafana 3000:80 --address 0.0.0.0 &
kubectl port-forward -n monitoring svc/kps-kube-prometheus-stack-prometheus
9090:9090 --address 0.0.0.0 &
```

Accès depuis le poste local via tunnel SSH :

```
ssh -N -L 3000:localhost:3000 -L 9090:localhost:9090 ubuntu@212.47.230.56
```

- Grafana : <http://localhost:3000> — login `admin / admin`
- Prometheus : <http://localhost:9090>



## Étape 2.3 — ServiceMonitor et vérification du scraping

Ajout de `templates/servicemonitor.yaml` dans le chart. Le label `release: kps` est obligatoire pour que le Prometheus de kube-prometheus-stack accepte ce ServiceMonitor :

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: backend-metrics
  labels:
    release: kps
spec:
  selector:
    matchLabels:
      app: webapp
      tier: back
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
  namespaceSelector:
    matchNames:
      - guestbook
```

Le port du Service backend doit être **nommé** (`name: http`) pour que la spec ServiceMonitor puisse le référencer, et les labels `app: webapp` / `tier: back` doivent être présents sur le Service :

```
# Dans backend-service.yaml
metadata:
  labels:
    app: webapp
    tier: back
ports:
  - port: 5000
    targetPort: 5000
    name: http
```

```
helm upgrade gb webapp-chart -n guestbook --values webapp-chart/values.yaml
kubectl get servicemonitor -n guestbook
```

```
# Vérification de l'endpoint /metrics
kubectl exec -n guestbook deploy/backend -- python3 -c \
  "import urllib.request;
  print(urllib.request.urlopen('http://localhost:5000/metrics').read().decode())" |
  head -20
```

**serviceMonitor/guestbook/backend-metrics/0** 1 / 1 up ●

| Endpoint                       | Labels                                                                                                                                                     | Last scrape    | State |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|-------|
| http://10.42.2.98:5000/metrics | container="backend" endpoint="http" instance="10.42.2.98:5000" job="backend-svc" namespace="guestbook" pod="backend-c648bd7c7-nxhsk" service="backend-svc" | 4.231s ago 5ms | UP    |

**serviceMonitor/monitoring/kps-grafana/0** 1 / 1 up ●

| Endpoint                        | Labels                                                                                                                                                               | Last scrape     | State |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-------|
| http://10.42.2.105:3000/metrics | container="grafana" endpoint="http-web" instance="10.42.2.105:3000" job="kps-grafana" namespace="monitoring" pod="kps-grafana-bdd8dcdf9-rhq66" service="kps-grafana" | 3.801s ago 66ms | UP    |

**serviceMonitor/monitoring/kps-kube-prometheus-stack-apiserver/0** 1 / 1 up ●

| Endpoint                           | Labels                                                                                                  | Last scrape       | State |
|------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------|-------|
| https://212.47.230.56:6443/metrics | endpoint="https" instance="212.47.230.56:6443" job="apiserver" namespace="default" service="kubernetes" | 19.671s ago 367ms | UP    |

**serviceMonitor/monitoring/kps-kube-prometheus-stack-coredns/0** 1 / 1 up ●

| Endpoint                       | Labels                                                                                                                                                                               | Last scrape     | State |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-------|
| http://10.42.0.18:9153/metrics | container="coredns" endpoint="http-metrics" instance="10.42.0.18:9153" job="coredns" namespace="kube-system" pod="coredns-t4dffb5f-mpx2" service="kps-kube-prometheus-stack-coredns" | 19.109s ago 6ms | UP    |

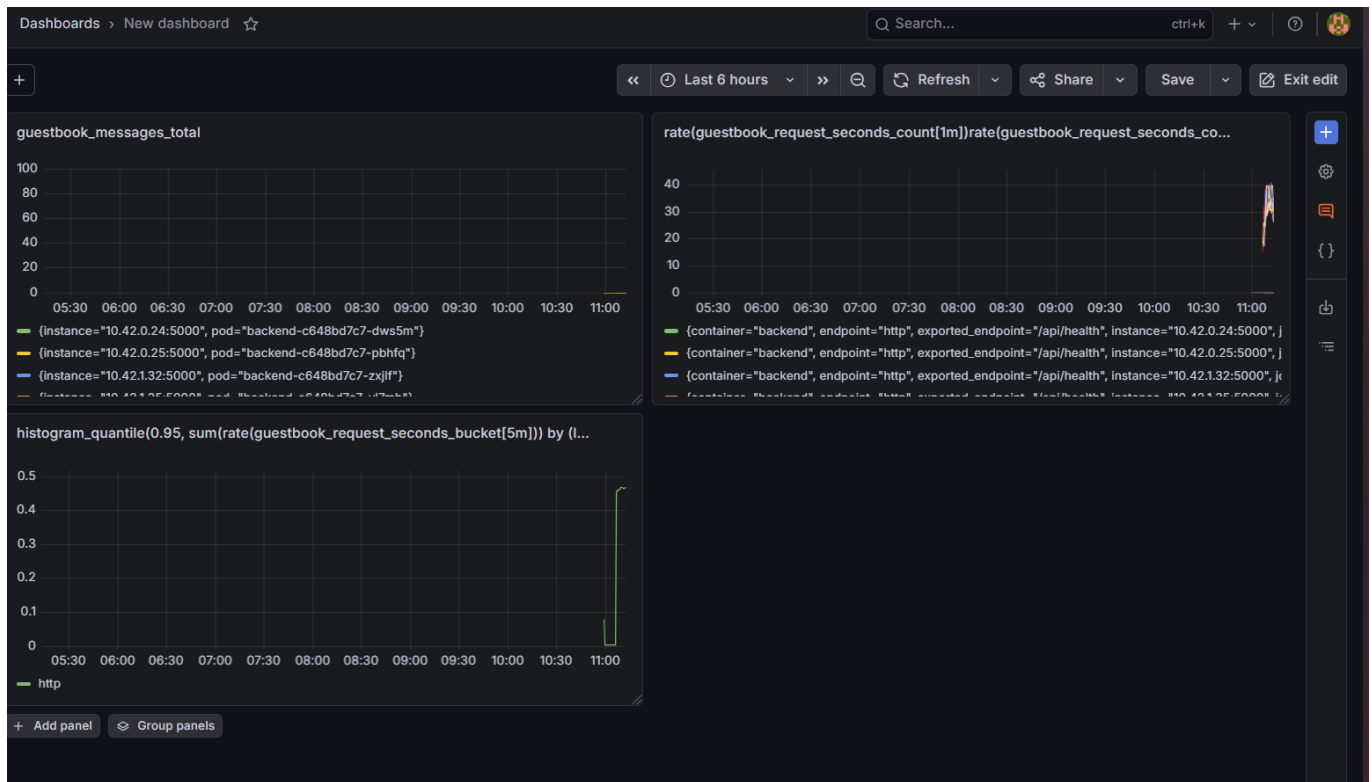
## Étape 2.4 — Requêtes PromQL et dashboard custom

Requêtes testées dans l'UI Prometheus :

```
guestbook_messages_total
rate(guestbook_request_seconds_count[1m])
histogram_quantile(0.95, sum(rate(guestbook_request_seconds_bucket[5m])) by (le, endpoint))
```

Dashboard Grafana créé avec trois panels :

- **Panel 1** — `guestbook_messages_total` : Time series, nombre cumulé de messages postés
- **Panel 2** — `rate(guestbook_request_seconds_count[1m])` : Time series par endpoint, trafic en temps réel
- **Panel 3** — `histogram_quantile(0.95, ...)` : p95 de latence par endpoint



**Piège rencontré :** le ServiceMonitor était bien créé mais Prometheus ne le détectait pas. Deux causes combinées : le label `release: kps` était absent, et les labels `app: webapp` / `tier: back` n'étaient pas présents sur le Service backend (Helm ne les injecte pas automatiquement). Vérifiable via `kubectl describe servicemonitor backend-metrics` et Prometheus UI → Status → Targets.

☒ **Checkpoint 2 :** Prometheus scrape le backend (UP dans Targets). Métrique `guestbook_messages_total` interrogeable. Dashboard Grafana custom opérationnel avec les 3 panels.

## Bloc 3 — HorizontalPodAutoscaler

### Objectif

Configurer un HPA sur le Deployment backend pour qu'il scale automatiquement entre 2 et 8 réplicas selon l'utilisation CPU, et le valider sous charge artificielle.

### Étape 3.1 — Vérification de metrics-server

k3s embarque `metrics-server` par défaut :

```
kubectl top nodes
kubectl top pods -n guestbook
```

```
ubuntu@GODON-k3s-server:~$ kubectl top nodes
NAME                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
godon-k3s-agent-1    39m          0%       914Mi           11%
godon-k3s-agent-2    156m         3%       1884Mi          23%
godon-k3s-server     100m         5%       1937Mi          24%
ubuntu@GODON-k3s-server:~$ kubectl top pods -n guestbook
NAME                CPU(cores)   MEMORY(bytes)
backend-c648bd7c7-nxhsk  2m           24Mi
frontend-69bbbcd85d-747sc 0m           4Mi
postgres-0           1m           54Mi
```

### Étape 3.2 — HPA sur CPU

Ajout de `templates/backend-hpa.yaml` dans le chart :

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2
  maxReplicas: 8
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
    policies:
    - type: Percent
      value: 100
      periodSeconds: 30
```

Le HPA nécessite que le Deployment ait des `resources.requests.cpu` définis — sans quoi il affiche `<unknown>/60%` et ne scale jamais.

```
helm upgrade gb webapp-chart -n guestbook --values webapp-chart/values.yaml
kubectl get hpa -n guestbook
```

```
kubectl describe hpa backend-hpa -n guestbook
```

```
ubuntu@GODON-k3s-server:~$ kubectl get hpa -n guestbook
```

| NAME        | REFERENCE          | TARGETS     | MINPODS | MAXPODS | REPLICAS | AGE |
|-------------|--------------------|-------------|---------|---------|----------|-----|
| backend-hpa | Deployment/backend | cpu: 2%/60% | 2       | 8       | 2        | 31s |

### Étape 3.3 — Test de charge

L'image [williamyeh/hey](#) n'étant pas compatible arm64, on utilise un Job Python natif :

```
apiVersion: batch/v1
kind: Job
metadata:
  name: load-test
  namespace: guestbook
spec:
  parallelism: 5
  completions: 5
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: load
        image: python:3.12-alpine
        command: ["python3", "-c"]
        args:
        - |
            import urllib.request, threading, time
            def worker():
                end = time.time() + 300
                while time.time() < end:
                    try:
                        urllib.request.urlopen('http://backend-
svc:5000/api/messages', timeout=2)
                    except:
                        pass
            threads = [threading.Thread(target=worker) for _ in range(50)]
            [t.start() for t in threads]
            [t.join() for t in threads]
```

```
kubectl apply -f load-test.yaml
```

Observation en temps réel dans deux terminaux séparés :

```
# Terminal 1
watch -n2 kubectl get hpa,deploy,pod -n guestbook
```

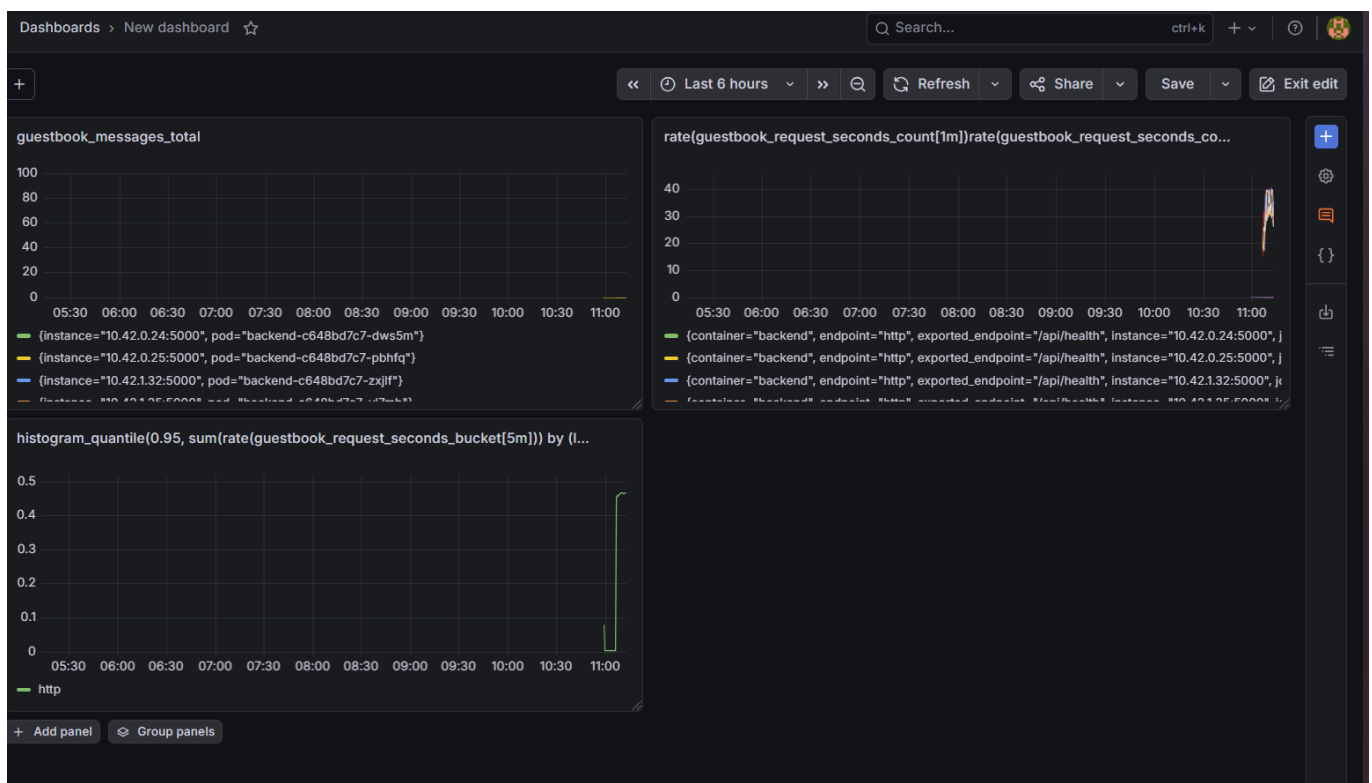
```
# Terminal 2
kubectl top pods -n guestbook
```

Séquence observée :

1. CPU backend monte au-dessus de 60%
2. HPA déclenche le scale-up : 2 → 4 → 8 réplicas
3. Sur Grafana, `rate(guestbook_request_seconds_count[1m])` explose
4. Après suppression du Job, la charge chute
5. Après 60s de stabilisation, scale-down à 2 réplicas

```
kubectl delete job load-test -n guestbook
```

```
Every 2.0s: kubectl get hpa -n guestbook
GODON-k3s-server: Mon May 11 09:07:31 2026
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa   Deployment/backend  cpu: 235%/60%    2         8         8          3m1s
```



```
ubuntu@GODON-k3s-server:~$ kubectl get hpa -n guestbook
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa   Deployment/backend  cpu: 2%/60%      2         8         2          12m
```

**Piège rencontré** : le HPA affichait `<unknown>/60%` au démarrage. Cause : les `resources.requests.cpu` n'étaient pas définis dans le template backend du chart. Ajouté dans `values.yaml` sous `backend.resources.requests.cpu: 50m`, puis `helm upgrade`.



**Piège rencontré :** l'image `williamyeh/hey` utilisée comme générateur de charge n'est pas compatible arm64. Remplacée par un Job Python utilisant `urllib.request` et `threading`, disponible nativement sur toutes les architectures.

☑ **Checkpoint 3 :** HPA passe le backend de 2 à 8 réplicas sous charge, visible dans `kubectl get hpa` et Grafana. Retour à 2 réplicas après la fenêtre de stabilisation de 60s.

## Bloc 4 — Pipeline CI/CD

### Objectif

Automatiser le cycle build → test → deploy via un pipeline GitHub Actions, de sorte qu'un `git push` sur `master` déclenche automatiquement un `helm upgrade` sur le cluster.

### Étape 4.1 — Architecture

```
[git push]
|
├─ build-backend : docker buildx build & push (multi-arch amd64+arm64)
├─ build-frontend : docker buildx build & push (multi-arch amd64+arm64)
├─ helm-lint : helm lint + helm template
└─ deploy : helm upgrade --install → cluster k3s
```

### Étape 4.2 — ServiceAccount dédié CI

Pour ne pas injecter un kubeconfig admin dans la CI, on crée un ServiceAccount `ci-deployer` limité au namespace `guestbook` :

```
kubectl apply -f ci-rbac.yaml
TOKEN=$(kubectl get secret ci-deployer-token -n guestbook -o
jsonpath='{.data.token}' | base64 -d)
CA=$(kubectl get secret ci-deployer-token -n guestbook -o
jsonpath='{.data.ca\.crt}')
```

Construction du kubeconfig CI avec l'IP publique du serveur et encodage :

```
cat > kubeconfig-ci.yaml <<EOF
apiVersion: v1
kind: Config
clusters:
- name: k3s
  cluster:
    server: https://212.47.230.56:6443
    certificate-authority-data: ${CA}
users:
- name: ci
```

```

user:
  token: ${TOKEN}
contexts:
- name: ci@k3s
  context:
    cluster: k3s
    user: ci
    namespace: guestbook
current-context: ci@k3s
EOF

```

```
base64 -w 0 kubeconfig-ci.yaml
```

```
# → Copier dans GitHub Settings → Secrets and variables → Actions → KUBECONFIG_B64
```

## Repository secrets

[New repository secret](#)

| Name             | Last updated   |                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  DOCKERHUB_TOKEN | 22 minutes ago |     |
|  DOCKERHUB_USER  | 24 minutes ago |     |
|  KUBECONFIG_B64 | 3 minutes ago  |   |

## Étape 4.3 — Le `.github/workflows/deploy.yml`

```

name: CI/CD

on:
  push:
    branches: [ master ]

permissions:
  contents: read

env:
  IMAGE_BACKEND: docker.io/${{ secrets.DOCKERHUB_USER }}/webapp-backend
  IMAGE_FRONTEND: docker.io/${{ secrets.DOCKERHUB_USER }}/webapp-frontend

jobs:
  build-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Login Docker Hub
        run: echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login -u "${{
secrets.DOCKERHUB_USER }}" --password-stdin
      - name: Build & push backend
        run: |
          docker buildx create --use

```

```

    docker buildx build \
      --platform linux/amd64,linux/arm64 \
      -t "${{ env.IMAGE_BACKEND }}:${{ github.sha }}" \
      --push ./TP/webapp/backend

build-frontend:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Login Docker Hub
      run: echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login -u "${{ secrets.DOCKERHUB_USER }}" --password-stdin
    - name: Build & push frontend
      run: |
        docker buildx create --use
        docker buildx build \
          --platform linux/amd64,linux/arm64 \
          -t "${{ env.IMAGE_FRONTEND }}:${{ github.sha }}" \
          --push ./TP/webapp/frontend

helm-lint:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Install Helm
      run: curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
    - name: Lint
      run: |
        helm lint TP/webapp-chart
        helm template TP/webapp-chart --values TP/webapp-chart/values.yaml > /tmp/rendered.yaml
        wc -l /tmp/rendered.yaml

deploy:
  runs-on: ubuntu-latest
  needs: [build-backend, build-frontend, helm-lint]
  steps:
    - uses: actions/checkout@v4
    - name: Install Helm
      run: curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
    - name: Setup kubeconfig
      run: |
        mkdir -p ~/.kube
        echo "${{ secrets.KUBECONFIG_B64 }}" | base64 -d > ~/.kube/config
        chmod 600 ~/.kube/config
    - name: Deploy
      run: |
        helm upgrade --install gb TP/webapp-chart \
          --namespace guestbook \
          --values TP/webapp-chart/values.yaml \
          --set global.registry=docker.io/${{ secrets.DOCKERHUB_USER }} \
          --set backend.tag=${{ github.sha }} \

```

```
--set frontend.tag=${{ github.sha }} \  
--wait --timeout 5m
```

Étape 4.4 — Démonstration du pipeline

On modifie le message de bienvenue dans `backend/app.py` et on pousse sur `master`. Séquence observée dans GitHub Actions :

- 1. `build-backend` → image taguée avec le SHA du commit, poussée sur Docker Hub (multi-arch)
- 2. `helm-lint` → succès
- 3. `deploy` → `helm upgrade`, rollout OK, livre d'or accessible avec la nouvelle image

```
# Vérification post-déploiement  
kubectl get pods -n guestbook  
kubectl describe deploy/backend -n guestbook | grep Image
```

← CI/CD

fix paths and add chart #3

Re-run all jobs Latest #2

Summary

All jobs

build-backend  
build-frontend  
helm-lint  
deploy

Run details  
Usage  
Workflow file

Re-run triggered 2 minutes ago

MDAprogra 9454594 master

Status

Success

Total duration

1m 57s

Artifacts

-

deploy.yml

on: push

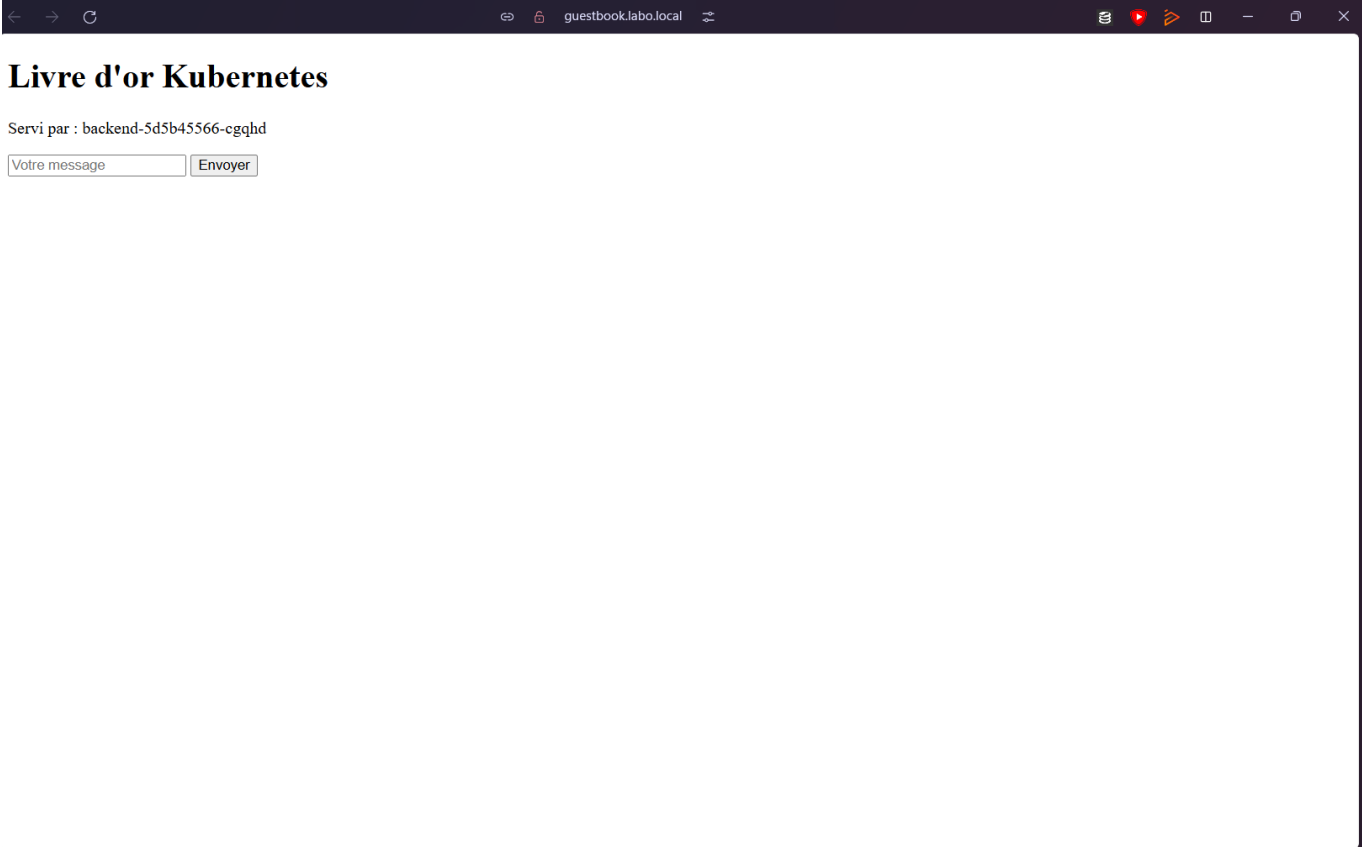
build-backend 56s

build-frontend 15s

helm-lint 7s

deploy 45s

| Description | Scope        | Status | Source | Created                  | Last used                | Expiration date       |
|-------------|--------------|--------|--------|--------------------------|--------------------------|-----------------------|
| GitHub      | Read & Write | Active | Manual | May 11, 2026 at 11:27:22 | May 11, 2026 at 11:47:23 | Jun 10, 2026 at 23:59 |



**Piège rencontré :** `helm upgrade` échouait avec `Kubernetes cluster unreachable: https://127.0.0.1:6443`. Le kubeconfig généré automatiquement via `kubectl config view` contenait `127.0.0.1` au lieu de l'IP publique. Solution : construire le kubeconfig manuellement avec `server: https://212.47.230.56:6443`.

**Bonne pratique :** ne jamais utiliser le mot de passe Docker Hub directement — utiliser un **PAT (Personal Access Token)** révoable créé sur `hub.docker.com`. Les secrets CI (`KUBECONFIG_B64`, `DOCKERHUB_TOKEN`) doivent être **masked** ET **protected** dans GitHub Actions.

☒ **Checkpoint 4 :** Pipeline GitHub Actions vert sur push. Image avec SHA visible sur Docker Hub. Application redéployée automatiquement, vérifiable via le navigateur.

### Synthèse des commandes TP3

| Commande                                                                      | Description                       |
|-------------------------------------------------------------------------------|-----------------------------------|
| <code>helm repo add &lt;nom&gt; &lt;url&gt;</code>                            | Ajouter un dépôt de charts        |
| <code>helm search repo &lt;terme&gt;</code>                                   | Chercher un chart dans les dépôts |
| <code>helm install &lt;release&gt; &lt;chart&gt; --values &lt;file&gt;</code> | Installer une release             |
| <code>helm upgrade &lt;release&gt; &lt;chart&gt; -n &lt;ns&gt;</code>         | Mettre à jour une release         |
| <code>helm rollback &lt;release&gt; &lt;revision&gt; -n &lt;ns&gt;</code>     | Revenir à une révision            |
| <code>helm history &lt;release&gt; -n &lt;ns&gt;</code>                       | Historique des révisions          |
| <code>helm lint &lt;chart&gt;</code>                                          | Vérification statique du chart    |

| Commande                                                       | Description                    |
|----------------------------------------------------------------|--------------------------------|
| <code>helm template &lt;chart&gt; --values &lt;file&gt;</code> | Rendu sans déploiement         |
| <code>helm uninstall &lt;release&gt; -n &lt;ns&gt;</code>      | Supprimer une release          |
| <code>kubectl top nodes</code>                                 | Consommation CPU/RAM des nœuds |
| <code>kubectl top pods -n &lt;ns&gt;</code>                    | Consommation CPU/RAM des pods  |
| <code>kubectl get hpa -n &lt;ns&gt;</code>                     | État de l'autoscaler           |
| <code>kubectl describe hpa &lt;nom&gt; -n &lt;ns&gt;</code>    | Détails et events du HPA       |

## Pièges rencontrés — TP3

| Symptôme                                                                     | Cause                                                                        | Résolution                                                                                     |
|------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>cannot re-use a name that is still in use</code>                       | <code>helm install</code> sur une release existante                          | Utiliser <code>helm upgrade</code> ou <code>helm uninstall</code> d'abord                      |
| Templates Helm cassés silencieusement                                        | <code>nindent</code> vs <code>indent</code> , quotes oubliées                | <code>helm template --debug</code> pour voir l'erreur exacte                                   |
| Prometheus n'a pas le target backend                                         | Label <code>release: kps</code> absent du ServiceMonitor                     | Ajouter <code>release: kps</code> dans les labels du ServiceMonitor                            |
| ServiceMonitor ignoré                                                        | Port du Service non nommé ou labels manquants sur le Service                 | Ajouter <code>name: http</code> sur le port et les labels <code>app/tier</code> sur le Service |
| HPA bloqué à <code>&lt;unknown&gt;/60%</code>                                | <code>resources.requests.cpu</code> absent du Deployment                     | Définir <code>requests.cpu</code> dans <code>values.yaml</code>                                |
| <code>ErrImagePull — no match for platform in manifest</code>                | Cluster multi-arch (amd64 + arm64) — image buildée pour une seule plateforme | Utiliser <code>docker buildx build --platform linux/amd64,linux/arm64 --push</code>            |
| Job de charge sans effet (image incompatible arm64)                          | <code>williamyeh/hey</code> non disponible sur arm64                         | Remplacer par un Job Python avec <code>urllib.request</code> et <code>threading</code>         |
| Pipeline <code>kubectl: connection refused</code> sur <code>127.0.0.1</code> | Kubeconfig généré avec l'IP locale au lieu de l'IP publique                  | Construire le kubeconfig manuellement avec <code>server: https://212.47.230.56:6443</code>     |
| <code>helm upgrade</code> timeout en CI                                      | Readiness probe trop serrée, Postgres lent                                   | Augmenter <code>initialDelaySeconds</code> , <code>-timeout 5m</code>                          |
| <code>docker push</code> denied en CI                                        | Mauvais PAT Docker Hub ou <code>DOCKERHUB_USER</code> incorrect              | Vérifier les secrets GitHub Actions, utiliser un PAT révocable                                 |